



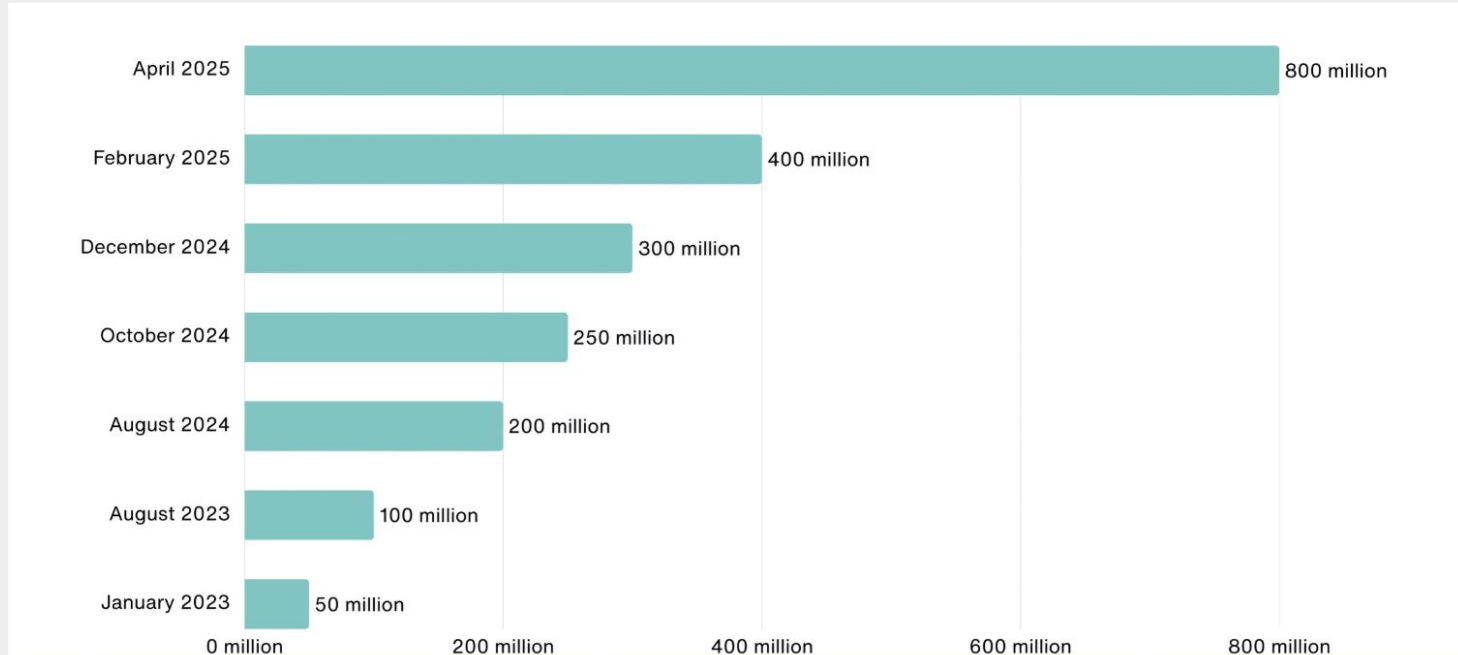
MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

NanoFlow & Mooncake

Presented by Albert Cao, John Xie, Eric Zhao, and Vishnu Kannan

Demand For Large-Scale Serving Systems

Number of weekly active ChatGPT users

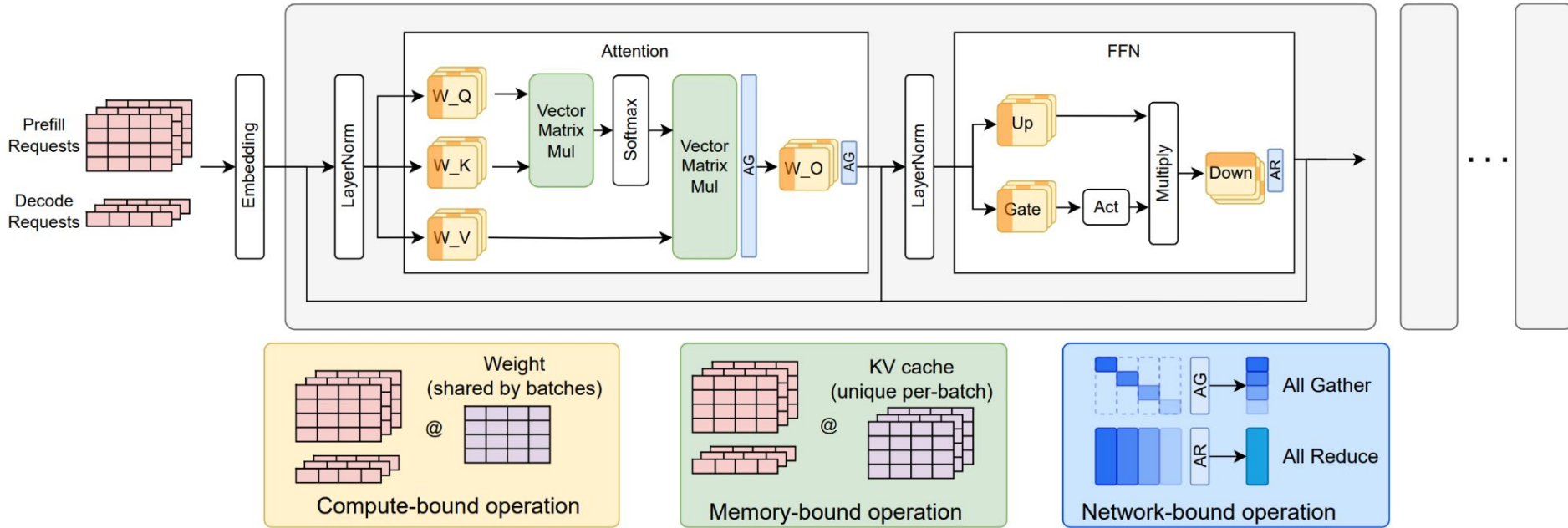


Maximizing Throughput

- Maximizing throughput is important because LLM serving is expensive
 - Higher throughput → can serve same amount of requests with fewer GPUs
- Two approaches for increasing throughput:
 1. NanoFlow: Optimize usage of bottlenecked resources
 2. Mooncake: Make tradeoffs between storage and computation

NanoFlow

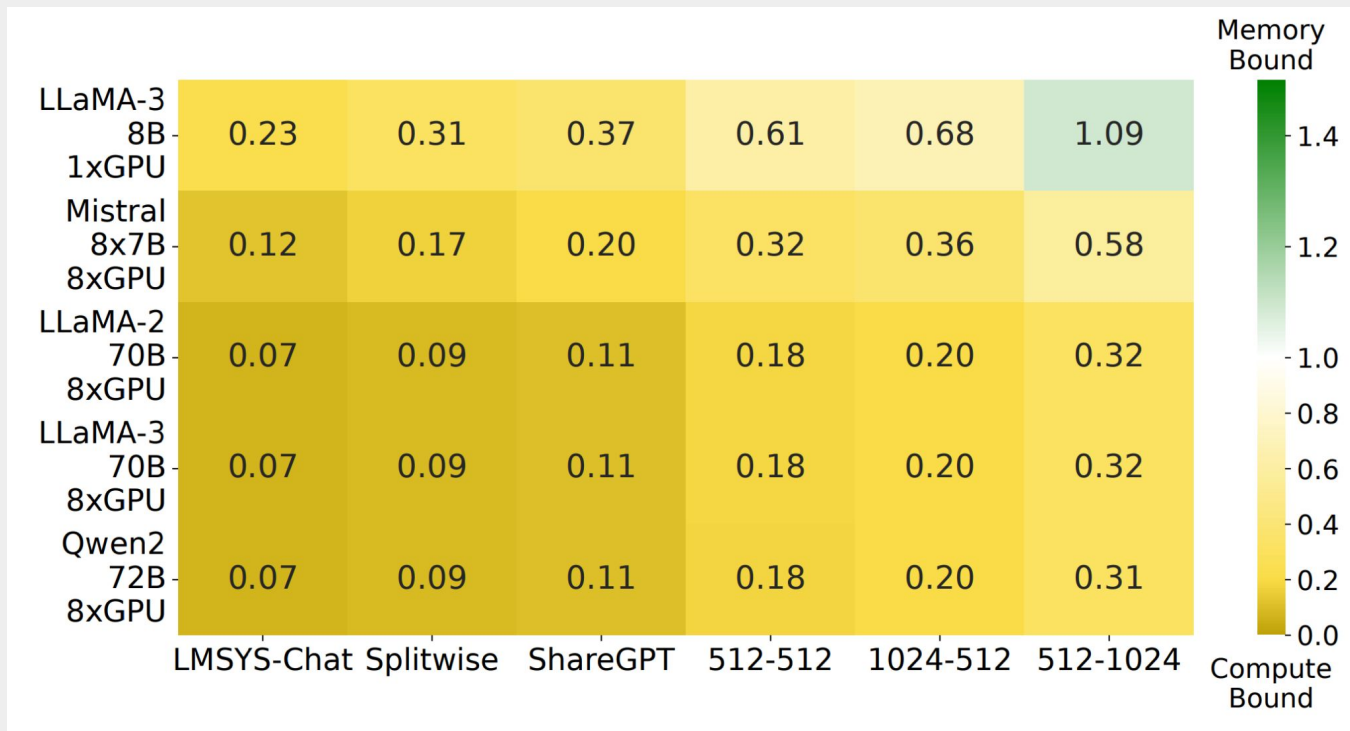
Decoder-Only Transformer



Theoretical Memory vs. Compute Time

$$\frac{T_{\text{Memory}}}{T_{\text{Compute}}}$$

Datasets	Avg. Input	Avg. Output
LMSYS-Chat	102	222
Splitwise	1155	211
ShareGPT	246	322



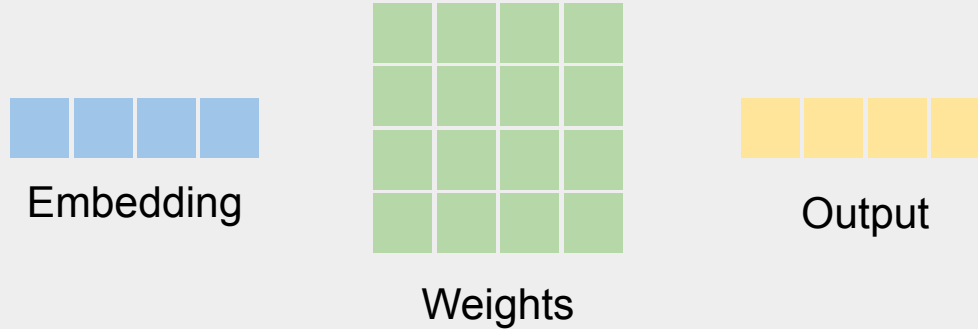
Theoretical Network vs. Compute Time

$$\frac{T_{\text{Network}}}{T_{\text{Compute}}}$$



Optimal Throughput when Compute Bound

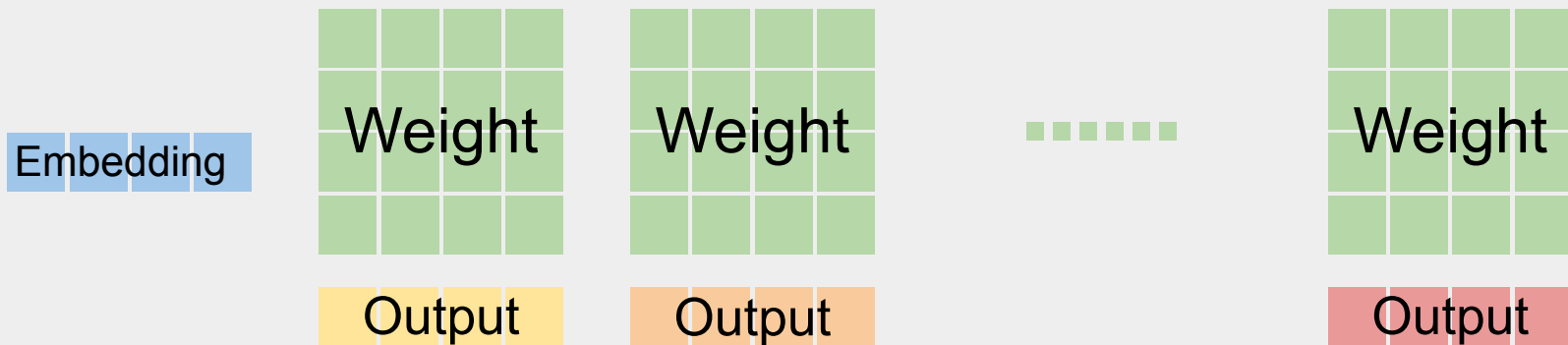
16 = Number of Parameters



16 Multiplication + 16 Addition = 32 total Floating Point Operations (FLOPs)

Optimal Throughput when Compute Bound

Number of Parameters = P_{model}



$$P_{\text{model}} \text{ (Mult)} + P_{\text{model}} \text{ (Add)} = 2P_{\text{model}} \text{ FLOPs}$$

$$\text{Throughput} = \frac{\text{GPU Compute}}{2P_{\text{model}}}$$

$$\text{Llama3 70B, A100: Optimal throughput} = \frac{280 \text{ TFLOPs}}{2 * 70B \text{ FLOPs}} = \mathbf{1857 \text{ tokens/s/GPU}}$$

Existing Frameworks vs. Optimal Throughput

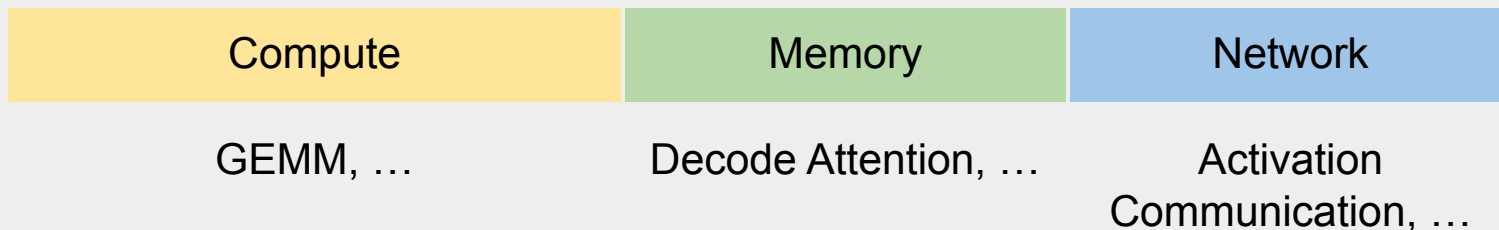
Optimal Throughput: 1857 tokens/s/GPU

Input/Output	vLLM	DeepSpeed-FastGen	TensorRT-LLM
512/512	26.60%	26.39%	39.58%
1024/512	29.73%	27.63%	44.00%
512/1024	22.08%	20.03%	34.25%

Compute/Memory/Network Bound Operations

- Doesn't achieve 100% compute utilization during memory/network bound operations
- Should aim to maximize end-to-end compute utilization

Most-Constrained Resource

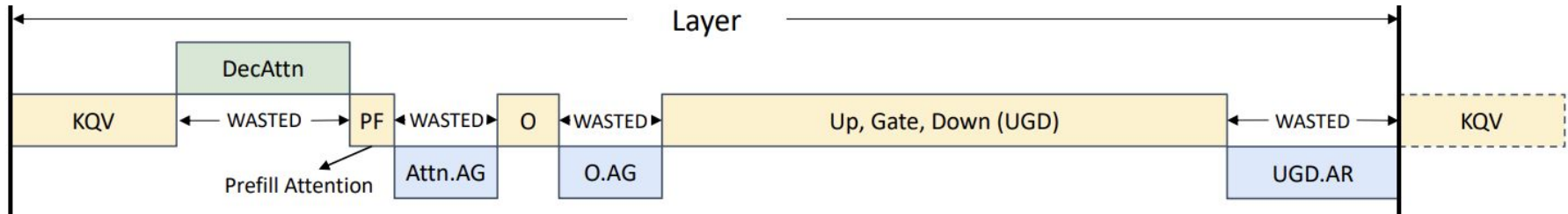


Example General Transformer Layer

Compute

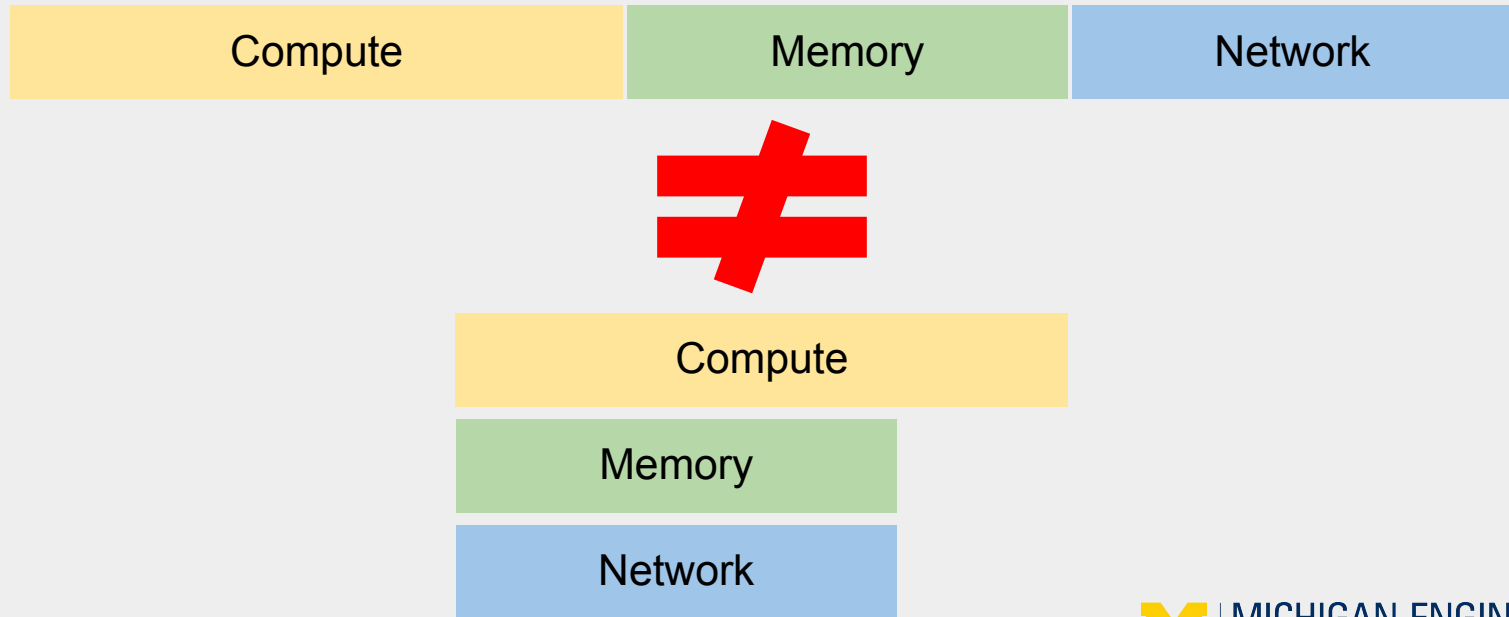
Memory

Network



No Direct Overlapping

- Because of interdependency, we cannot directly overlap operations



Splitting Operations

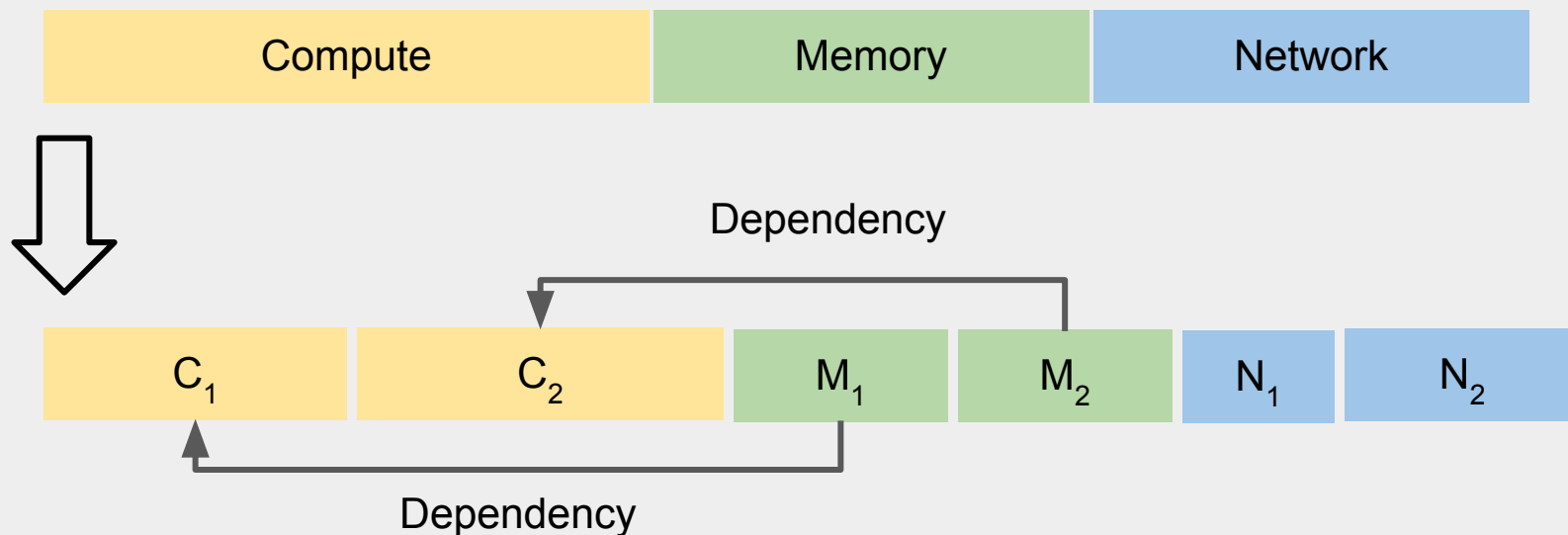
Nano-Batches:

B_1

B_2

Example: Request Batch = 1024 tokens $\rightarrow$ Nano-Batches = 512 + 512 tokens

1. Split up the compute/memory/network operations



Overlap Operations

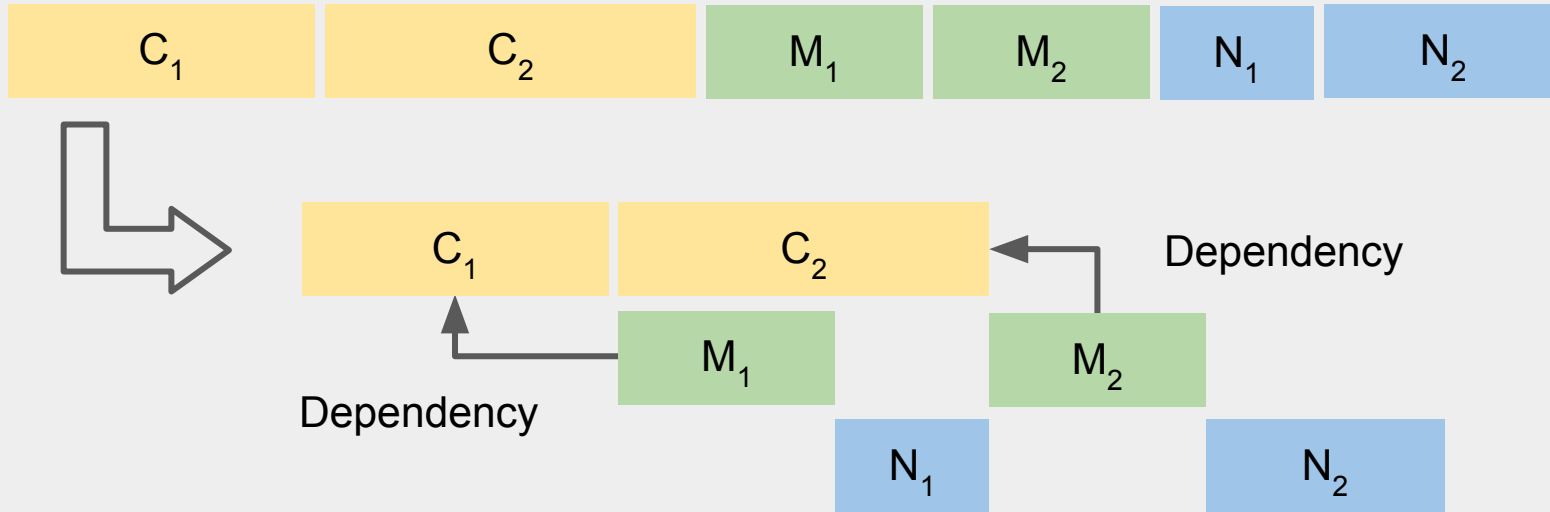
Nano-Batches:

B_1

B_2

Example: Request Batch = 1024 tokens $\rightarrow$ Nano-Batches = 512 + 512 tokens

2. Overlap operations that aren't dependent on each other

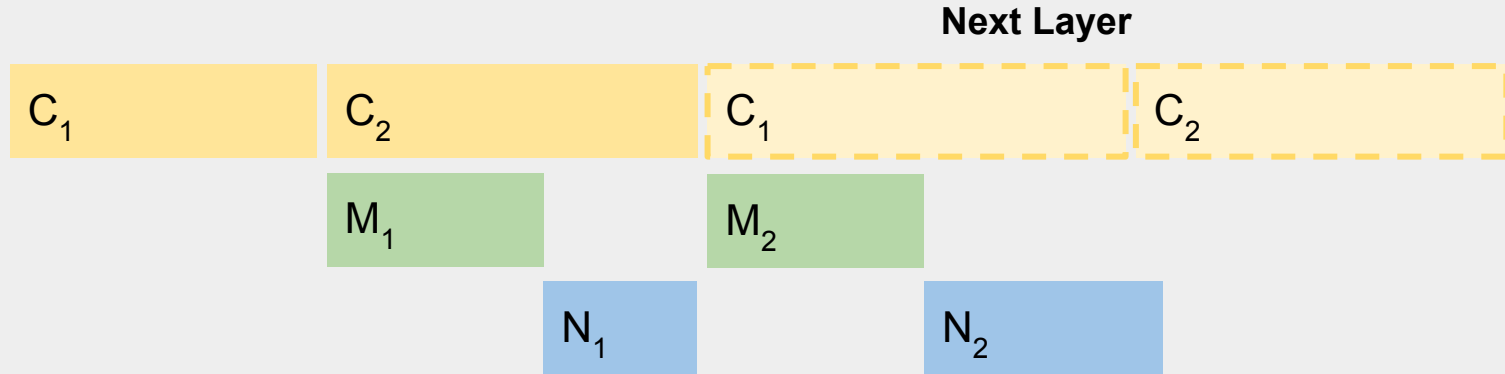


Batch Progression

Nano-Batches:



Example: Request Batch = 1024 tokens $\rightarrow$ Nano-Batches = 512 + 512 tokens

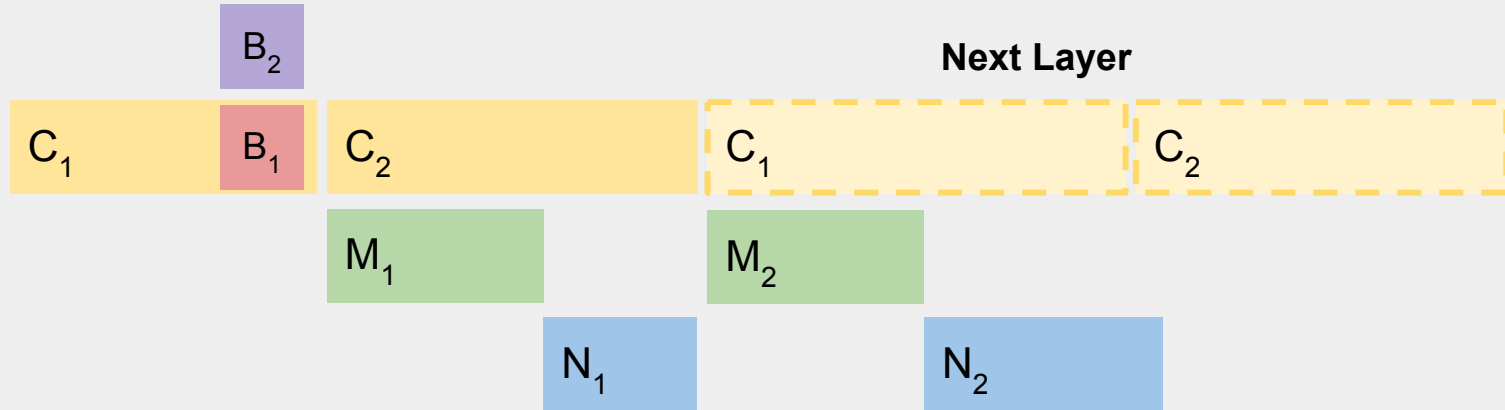


Batch Progression

Nano-Batches:



Example: Request Batch = 1024 tokens $\rightarrow$ Nano-Batches = 512 + 512 tokens

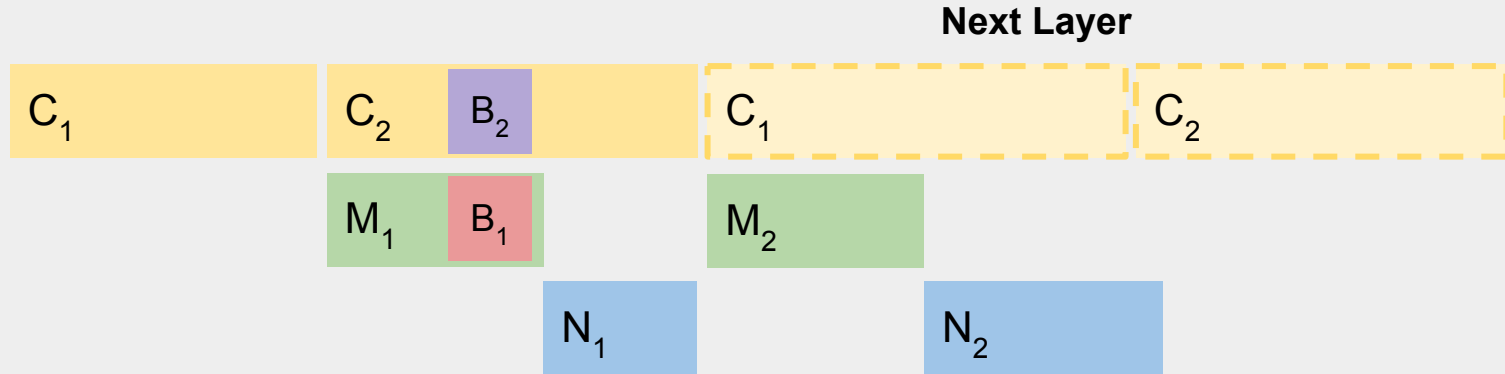


Batch Progression

Nano-Batches:



Example: Request Batch = 1024 tokens $\rightarrow$ Nano-Batches = 512 + 512 tokens

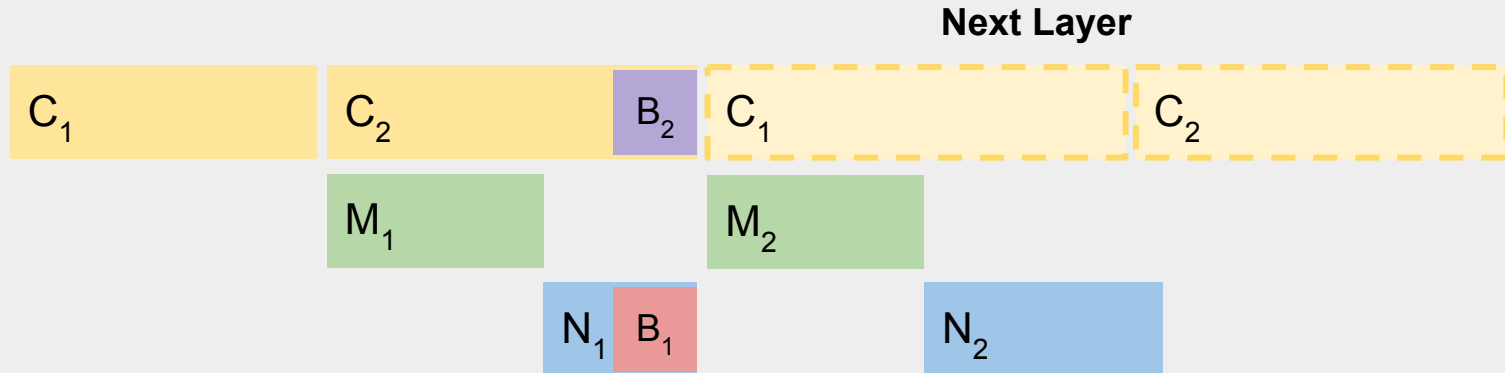


Batch Progression

Nano-Batches:



Example: Request Batch = 1024 tokens $\rightarrow$ Nano-Batches = 512 + 512 tokens

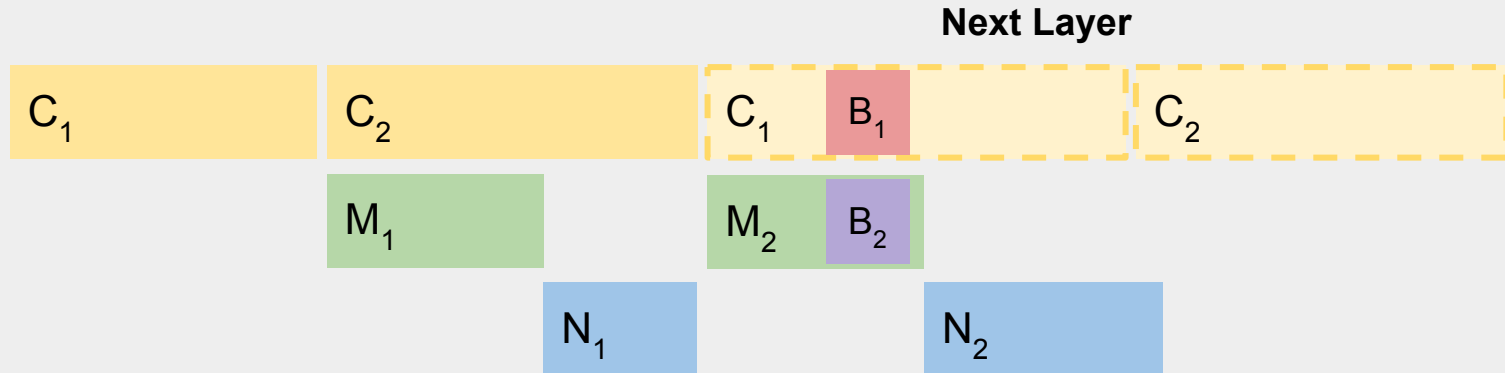


Batch Progression

Nano-Batches:



Example: Request Batch = 1024 tokens $\rightarrow$ Nano-Batches = 512 + 512 tokens

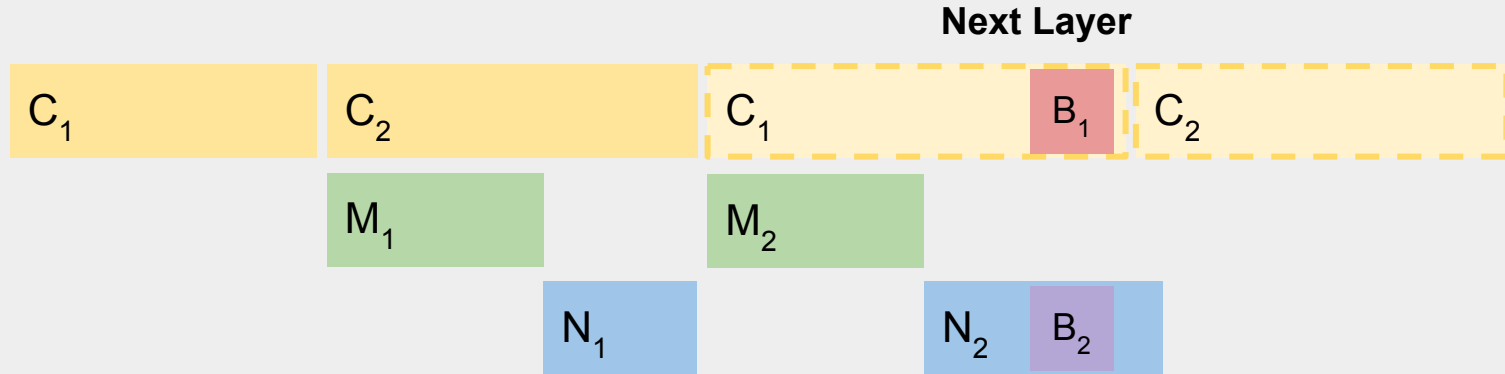


Batch Progression

Nano-Batches:



Example: Request Batch = 1024 tokens $\rightarrow$ Nano-Batches = 512 + 512 tokens



Two Stage Pipeline Scheduler

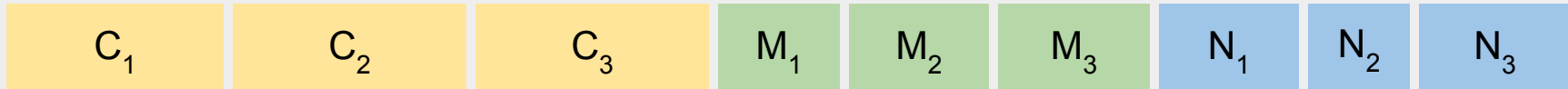
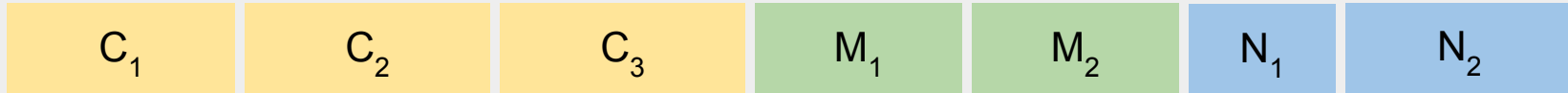
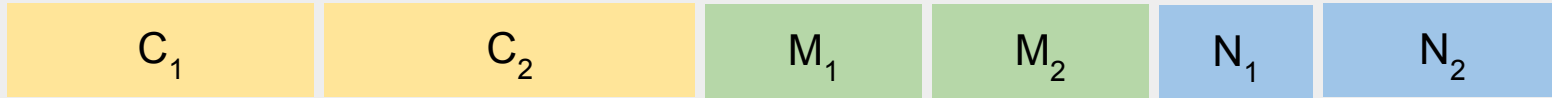
Phase I:

- Creates rough draft schedule that minimizes pipeline bubbles
- Does NOT account for overlap performance degradation
- Takes around 10 minutes

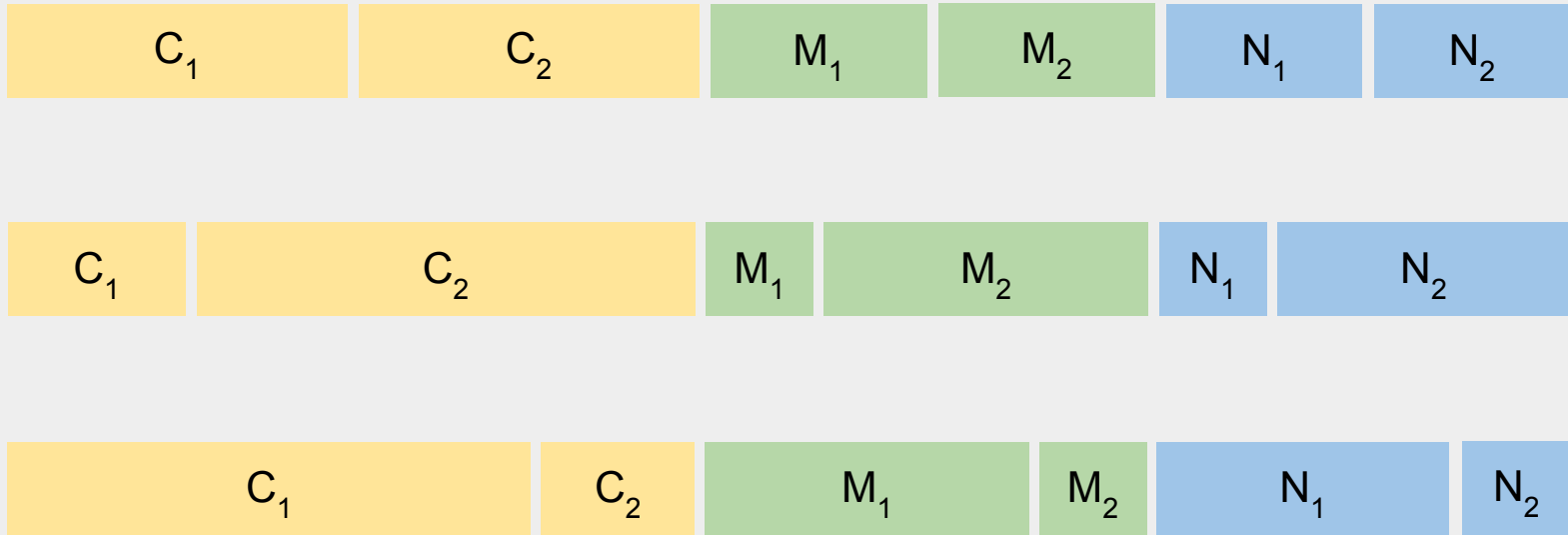
Phase II:

- Refines schedule that minimizes execution time
- Adds resource allocation to account for performance degradation
- Takes around 30 minutes

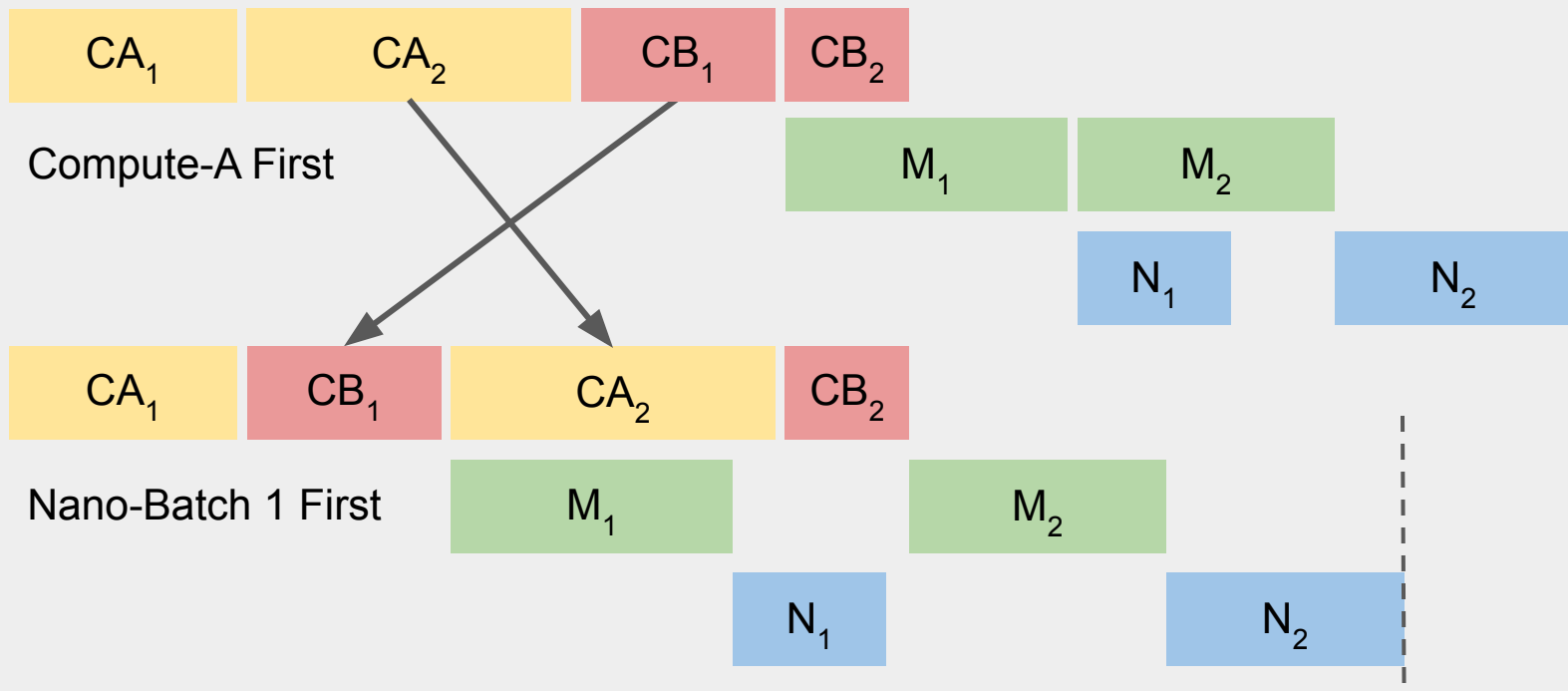
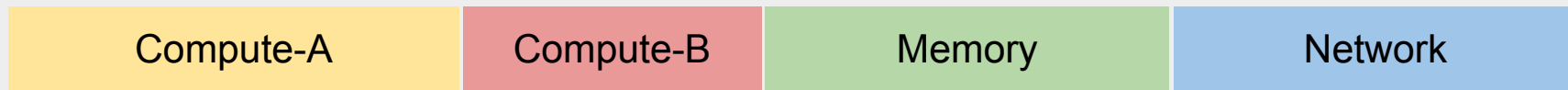
Number of Nano-Batches



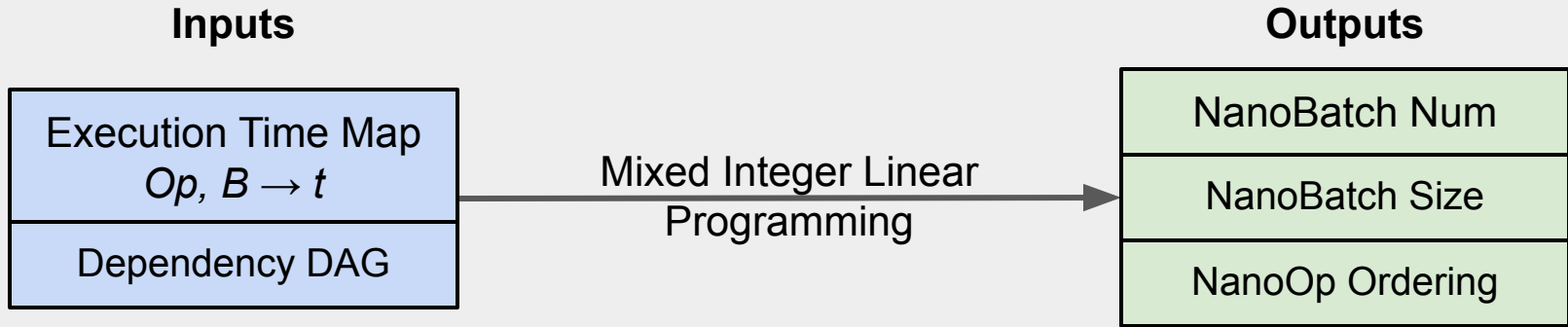
Size of Nano-Batches



Ordering of Nano Operations



Pipeline Phase I



8B Pipeline Design Example

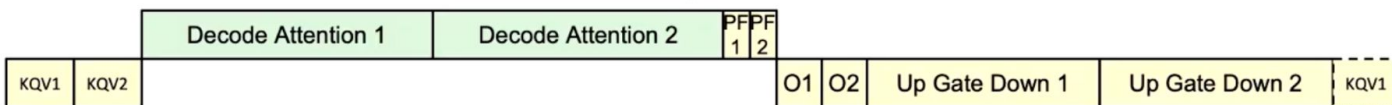
8B Pipeline Design

Compute-bound

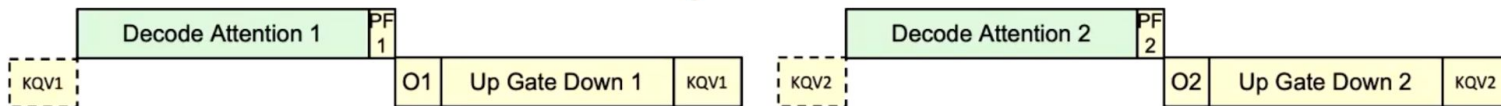
Memory-bound



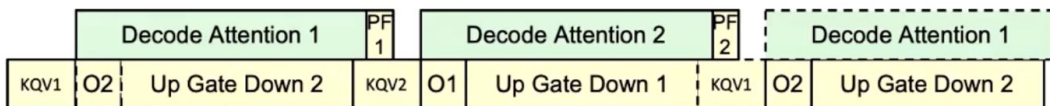
Split



Reorder



Overlap



Overlapping Operations Degrades Performance

- Generated “rough draft” of optimal solution that assume zero performance degradation when overlapping these operations
- However, because of GPU resource allocation, there are interference between kernels which degrades performance

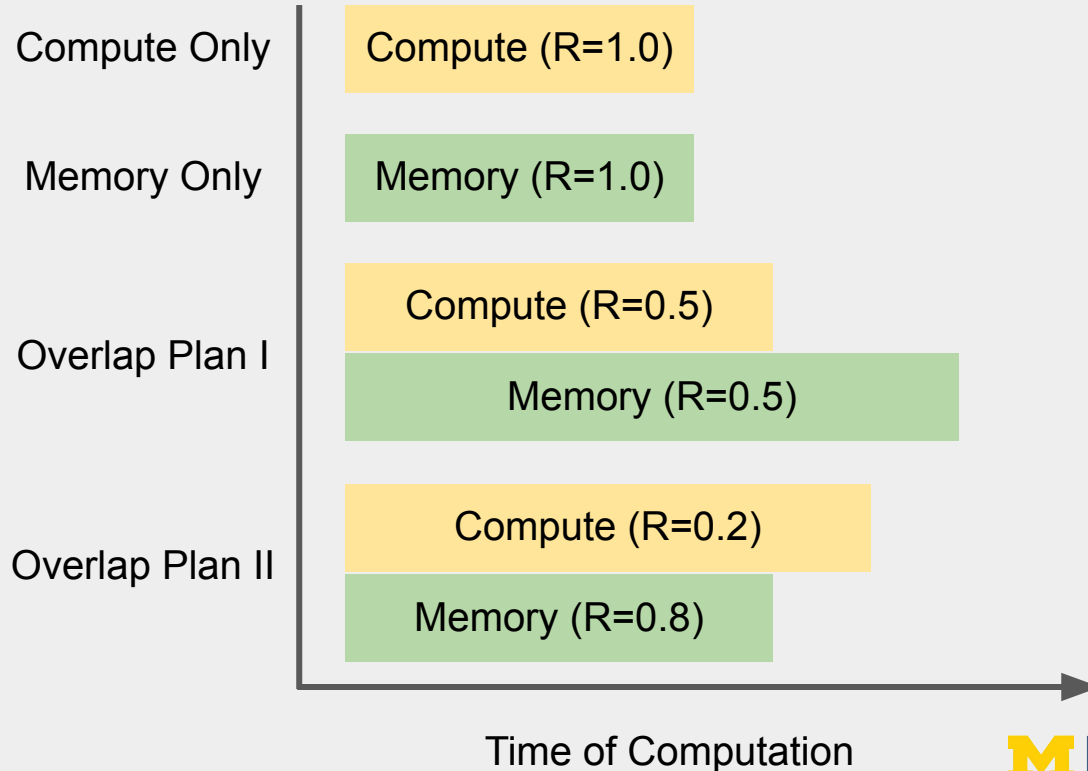
Performance Based on Resource Allocation

R = Proxy for a fractional allocation of a single GPU's resources

P = Performance when receiving R of the GPU normalized to peak performance

Operations	Resource Utilization (R)						
	0	0.1	0.2	...	0.8	0.9	1
GEMM (by definition)	0	0.1	0.2	...	0.8	0.9	1
GEMV	0	0.2	0.3	...	0.85	0.95	1
Network	0	0.3	0.5	...	0.9	1	1

Performance Degradation



Pipeline Phase II

Inputs

Execution Time Map $Op, B, R \rightarrow t$
Dependency DAG
NanoBatch Num
NanoBatch Size
NanoOp Ordering

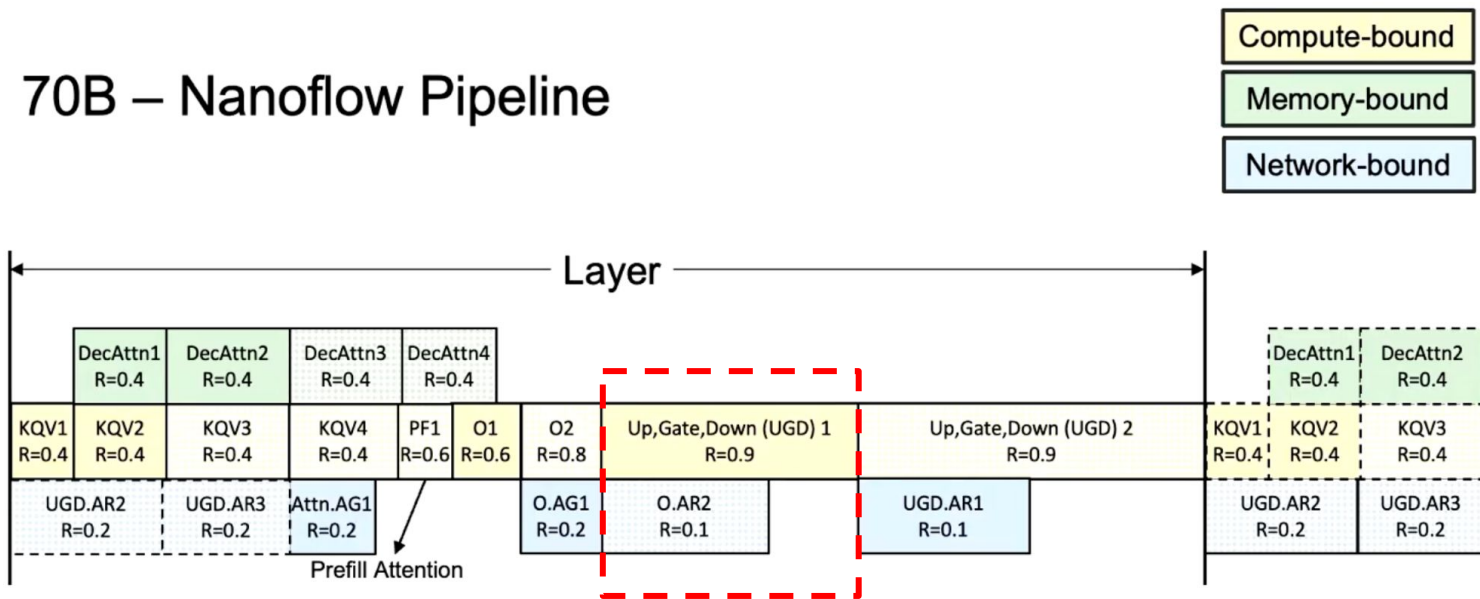
Mixed Integer Linear
Programming

Outputs

Resource Allocation R

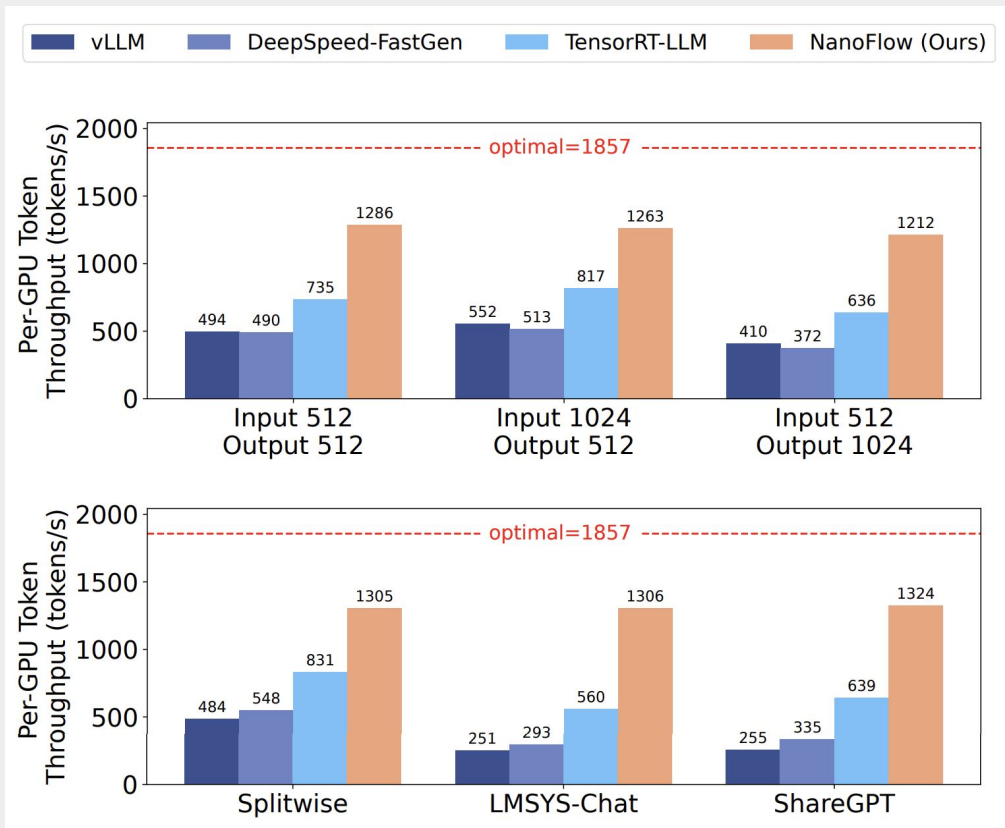
70B NanoFlow Pipeline Example

70B – Nanoflow Pipeline

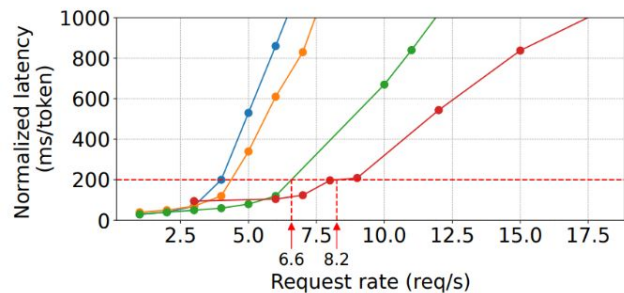


“Up, Gate, Down” requires more resources

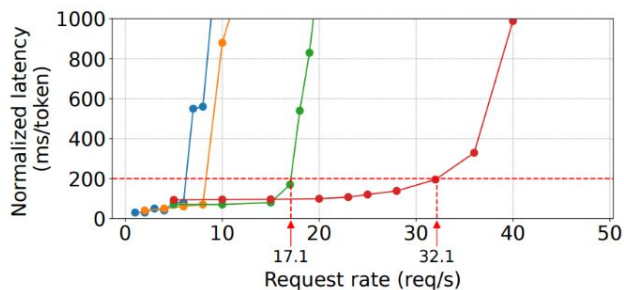
End-to-End Throughput Evaluation



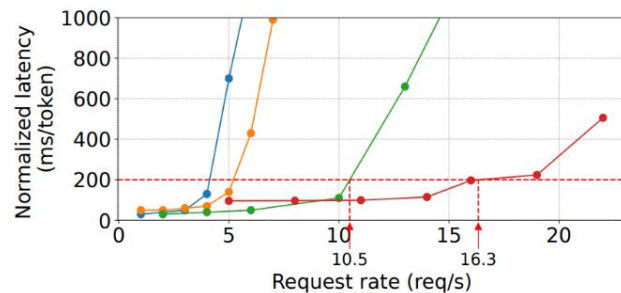
Latency Evaluation



(a) Splitwise



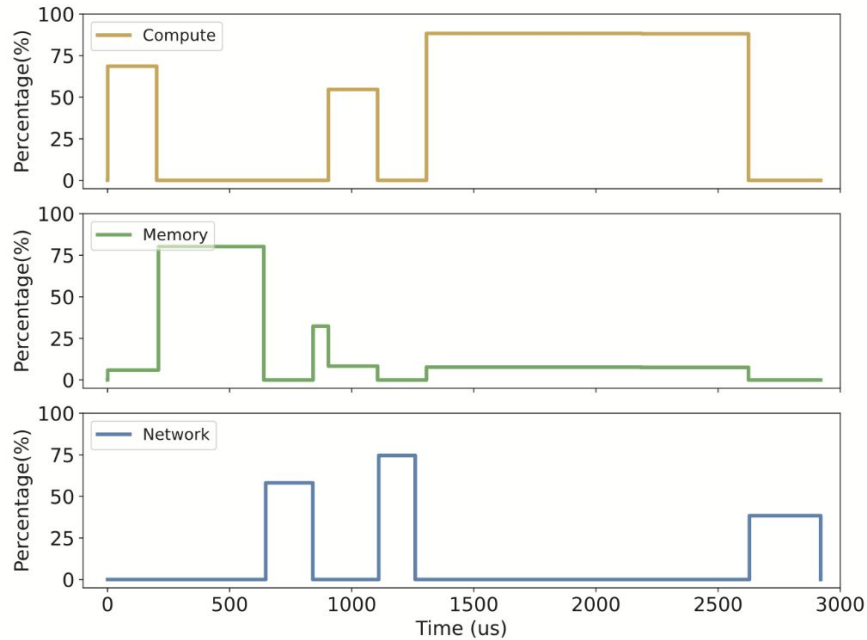
(b) LMSYS-Chat-1M



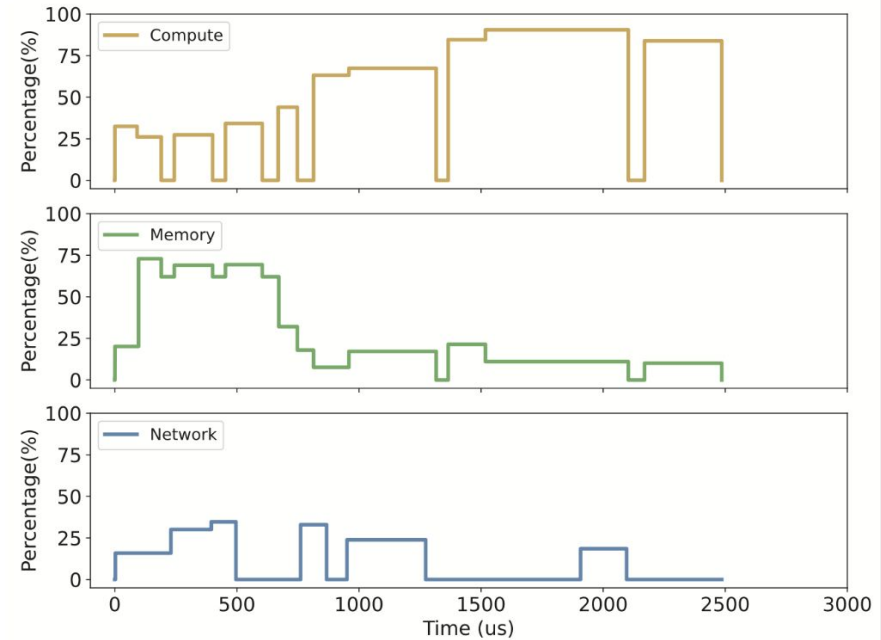
(c) ShareGPT

Resource Usage Evaluation

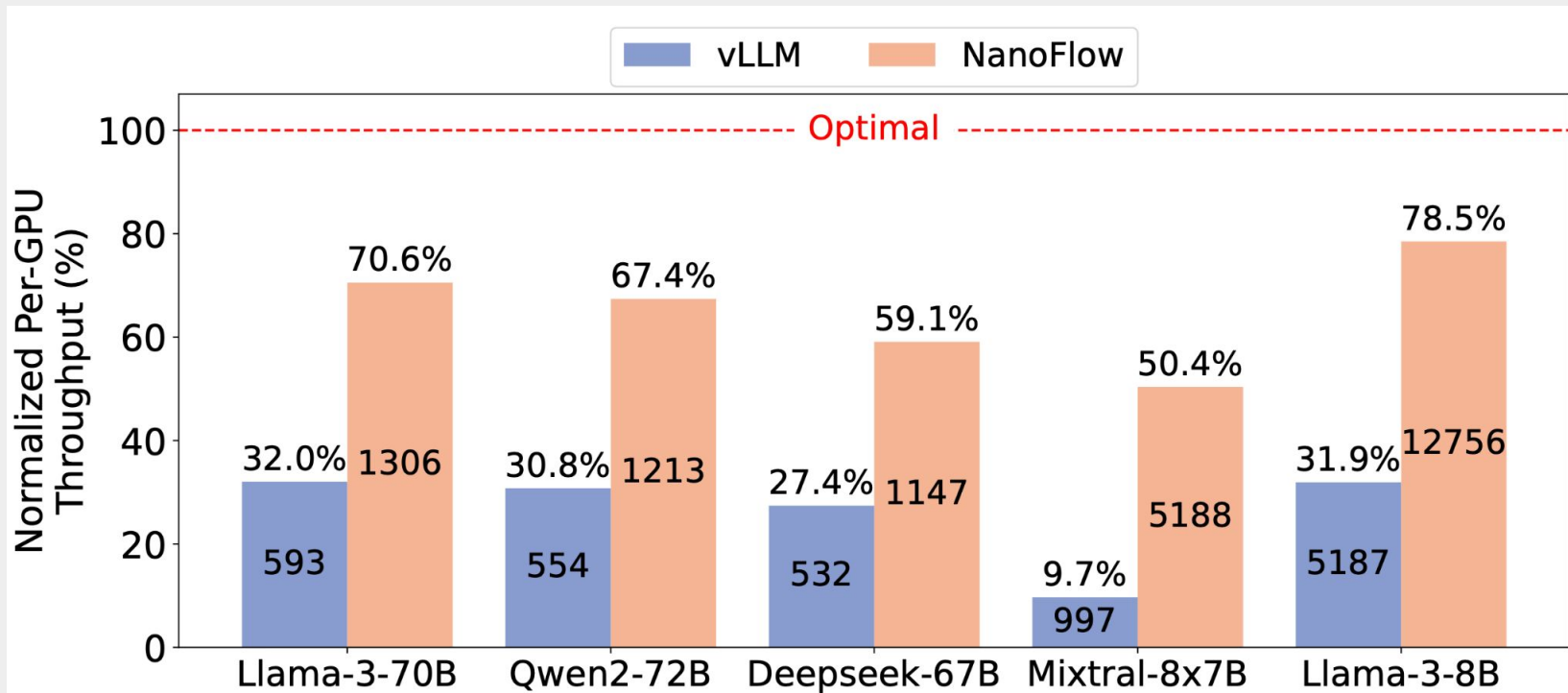
Baseline



NanoFlow



Evaluation on Different Models



NanoFlow Limitations

1. Both auto-search phases require profiling every kernel combination and solving MILPs
2. Pairwise interference profiling assumes performance degradation holds similar patterns for overlap between 2 operations and overlap between 3+ operations

Future Work

- Combine this intra-parallelism structure with other forms of parallelism to further increase throughput
- Heterogenous Support - CPU-GPU/GPU-GPU Cooperation
- The paper is too throughput focused, need to test other factors, such as energy efficiency, thermal throttling, per-request latency fairness

Mooncake

Mooncake: Overview

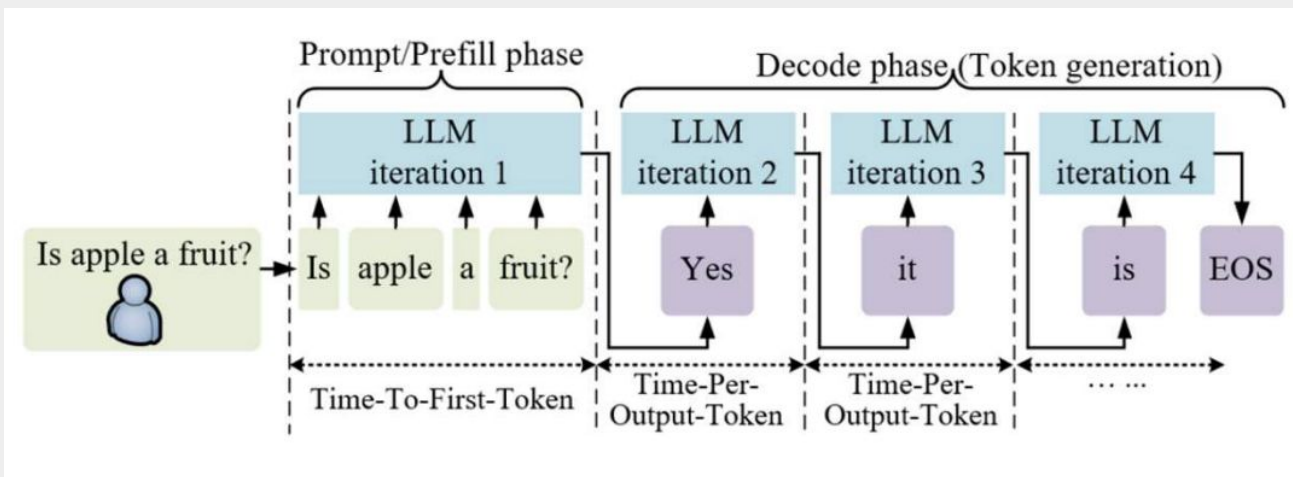
- Serving platform for Kimi developed by Moonshot AI
- Key characteristics:
 - Separate prefill and decode clusters
 - Global KVCache
- Design goals:
 - Maximize throughput
 - Adhere to latency-related service level objectives (SLOs)
- Experimental results:
 - Increases effective request capacity up to 498%
 - Handles 115% more requests on NVIDIA A800
 - Handles 107% more requests on NVIDIA H800

Mooncake: Background

Service Level Objectives

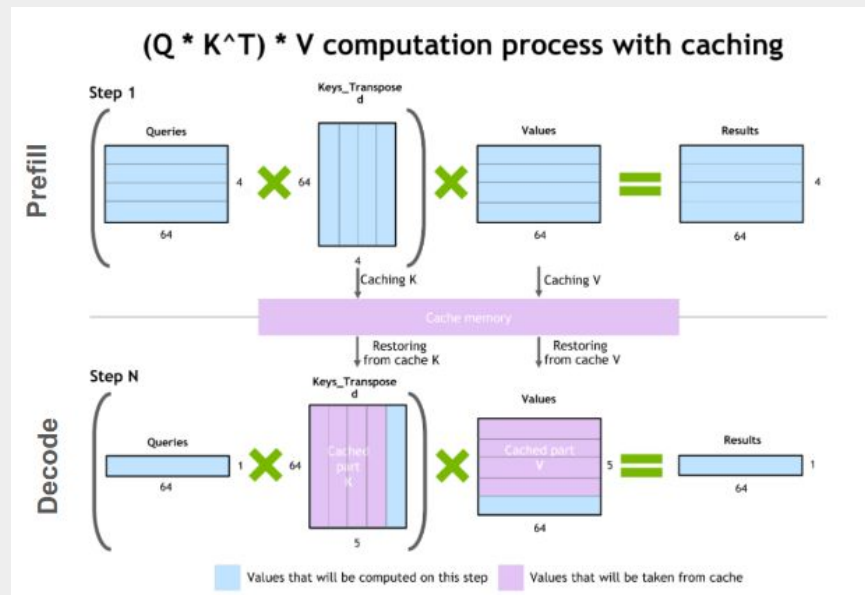
Time to first token (TTFT)

Time between tokens (TBT)



Mooncake: Background

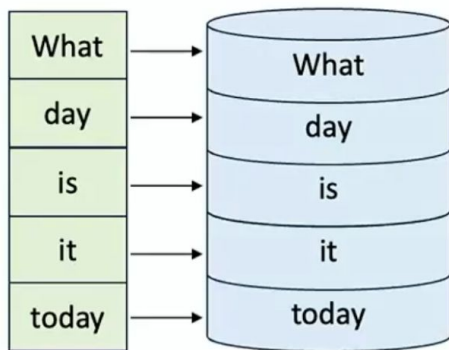
- **Current KVCache**
 - Per GPU and request
- **Local prefix caching (existing)**
 - Reuse KVCache across requests, per GPU
- **Global prefix caching (new)**
 - Reuse KVCache across **GPUs and requests**
 - Reduce computation cost during prefill



Prefix Caching

"What day is it today"

Prefill for Request I



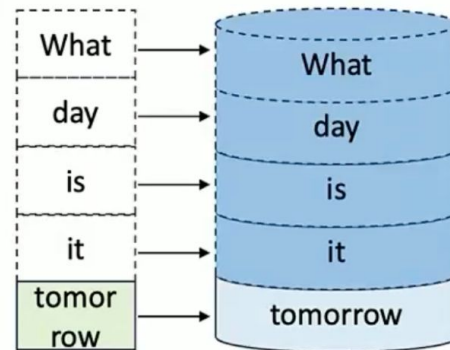
(5 tokens computed)

KVCache Reuse



"What day is it tomorrow"

Prefill for Request II



(1 token computed)

Why Disaggregated Architecture?

- Separate prefill and decode clusters (DistServe)
 - The two stages have fundamentally different resource constraints
 - Prevents interference between prefill and decode

Mooncake: Architecture

Conductor

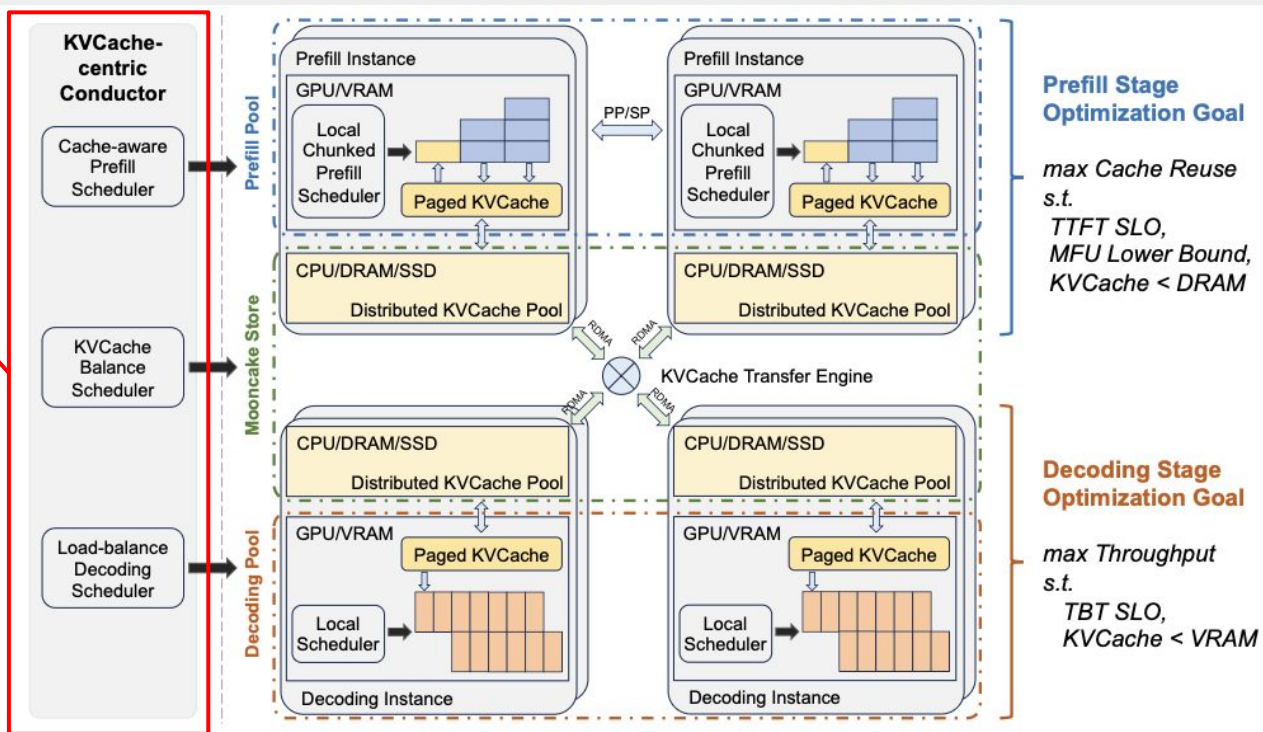
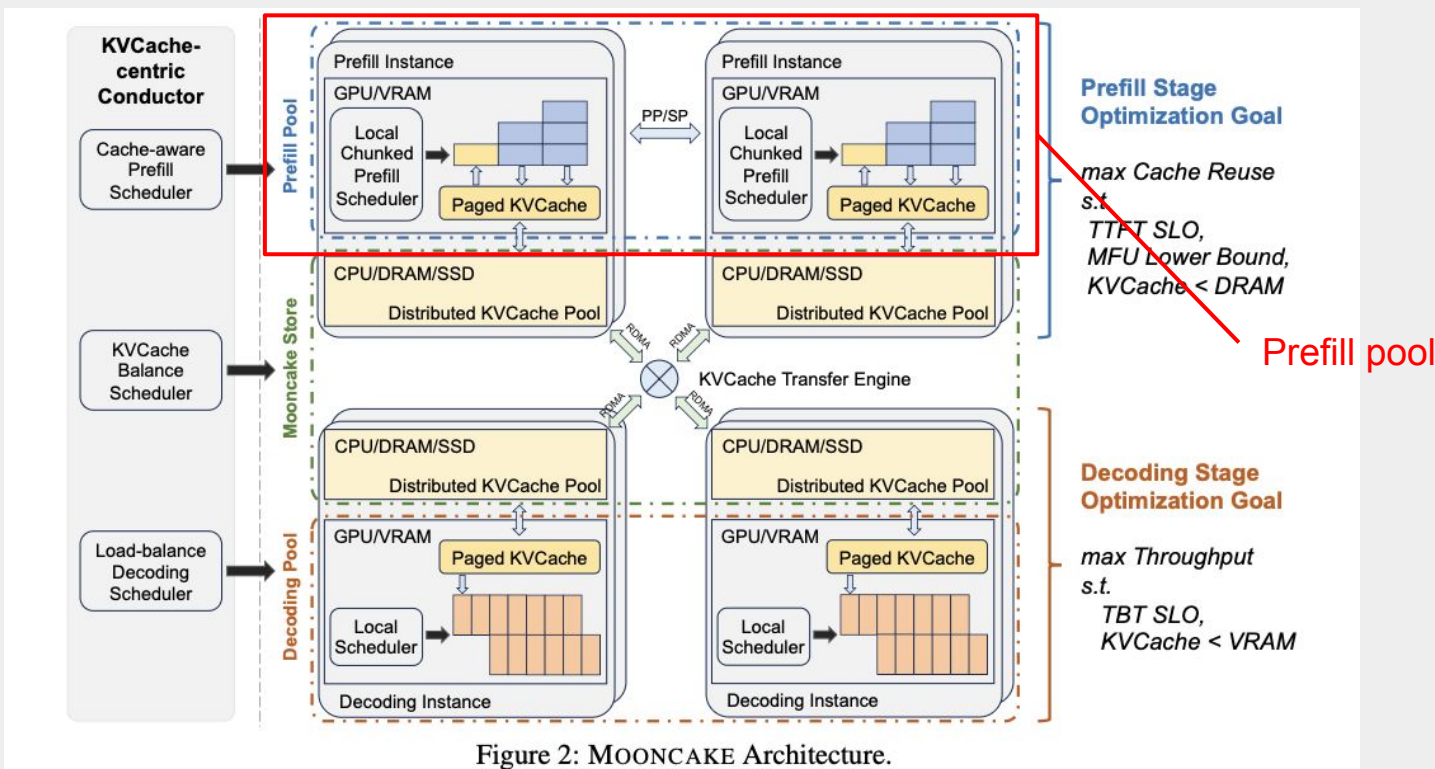


Figure 2: MOONCAKE Architecture.

Mooncake: Architecture



Mooncake: Architecture

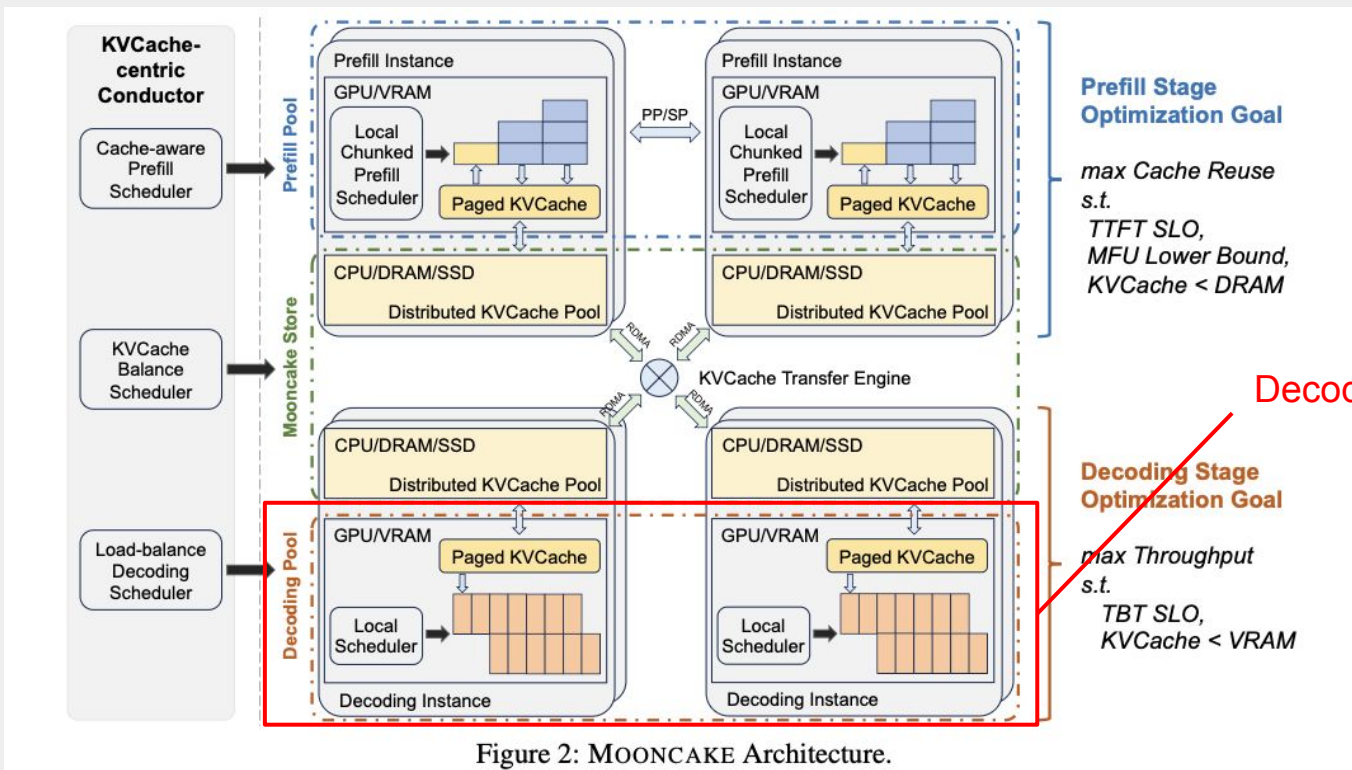


Figure 2: MOONCAKE Architecture.

Mooncake: Architecture

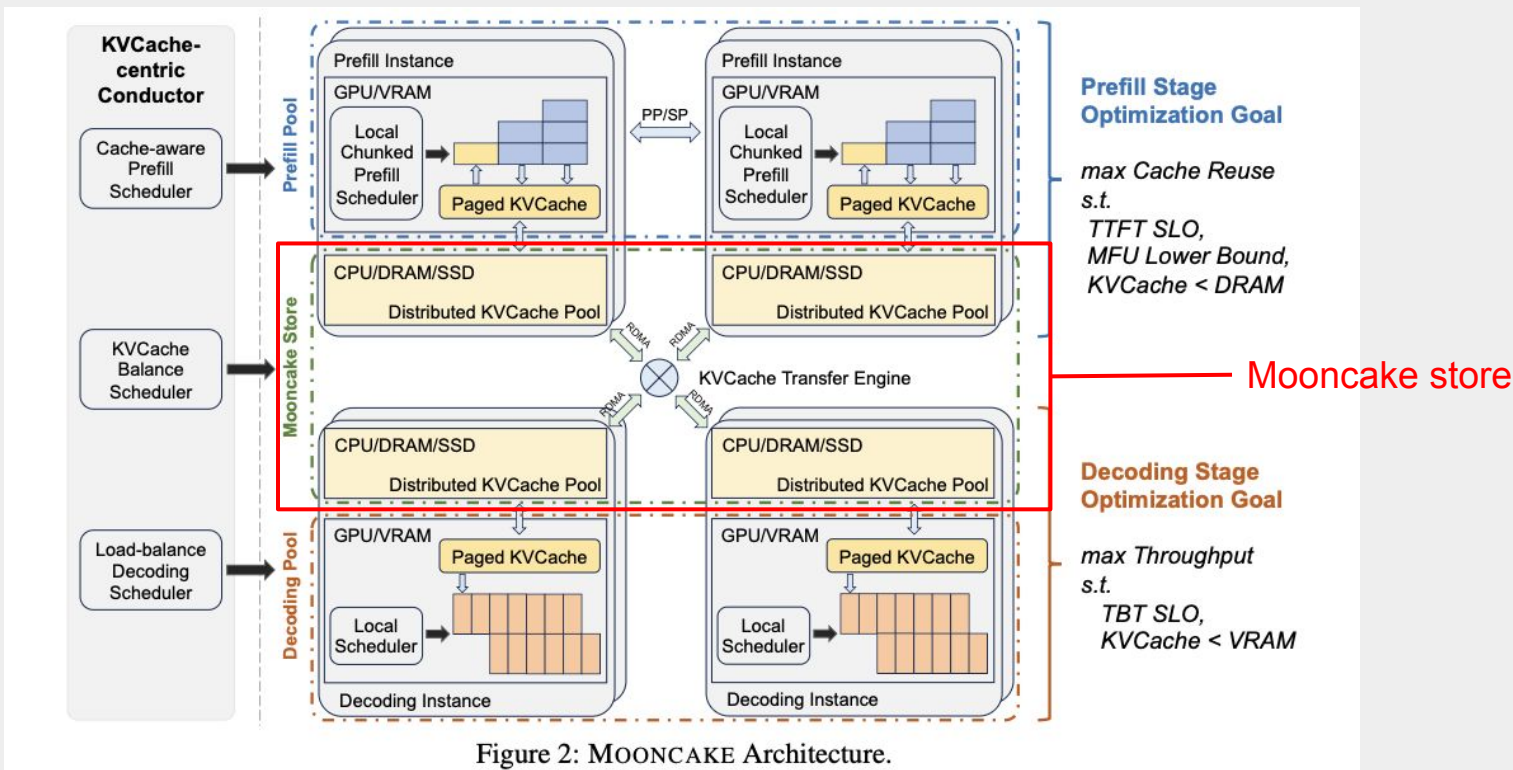


Figure 2: MOONCAKE Architecture.

Request Workflow

1. KVCache Reuse
 - a. Node receives request
2. Incremental Prefill
 - a. Computes prefill with prefix cache
3. KVCache Transfer
 - a. Asynchronous and overlaps with (2)
 - b. Stream KVCache to decode node per layer
4. Decoding
 - a. Node selected by conductor

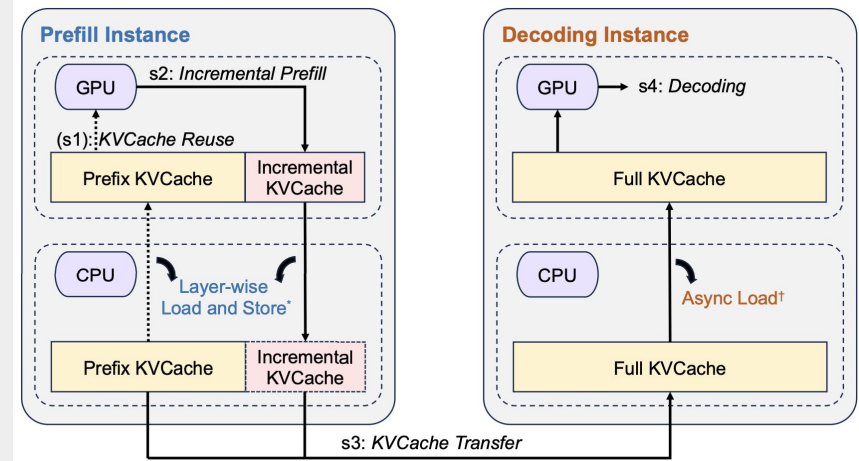
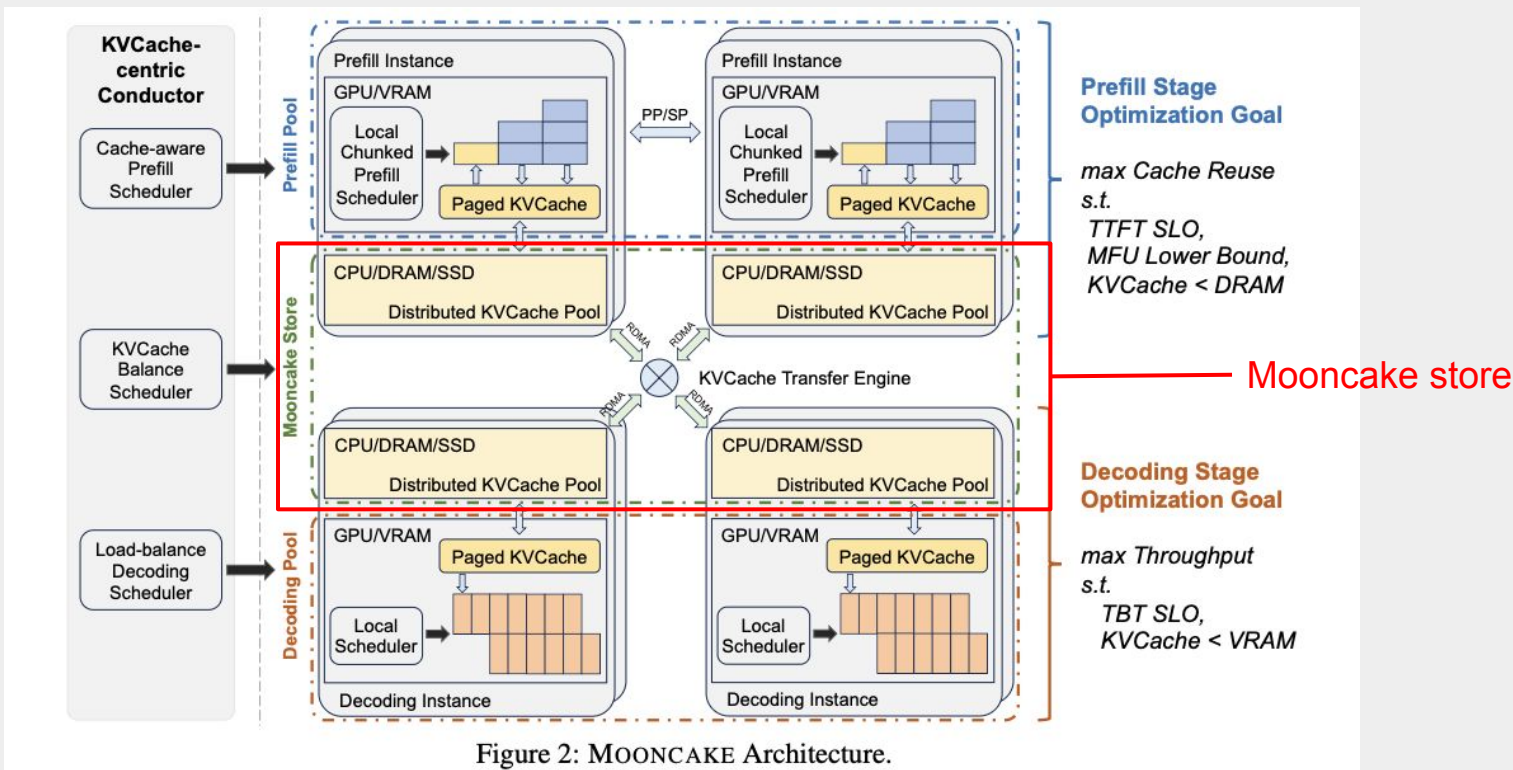


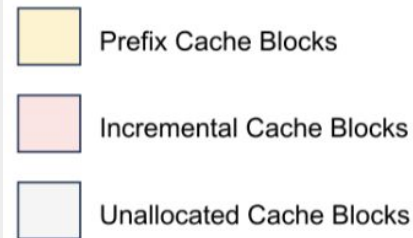
Figure 3: Workflow of inference instances.

Mooncake: Architecture



Mooncake Store: Cache of KVCache

$\text{Hash}(\text{block}) = \text{Hash}(\text{Hash}(\text{previous block}) + \text{current uncached token})$



Token Blocks
(16~512 tokens)

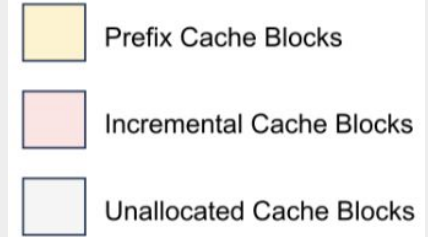
a	b	c	d	e	f	g	h	i
---	---	---	---	---	---	---	---	---

A=Hash(a)	B=Hash(A+b)	C=Hash(B+c)	D=Hash(C+d)	E=Hash(D+e)

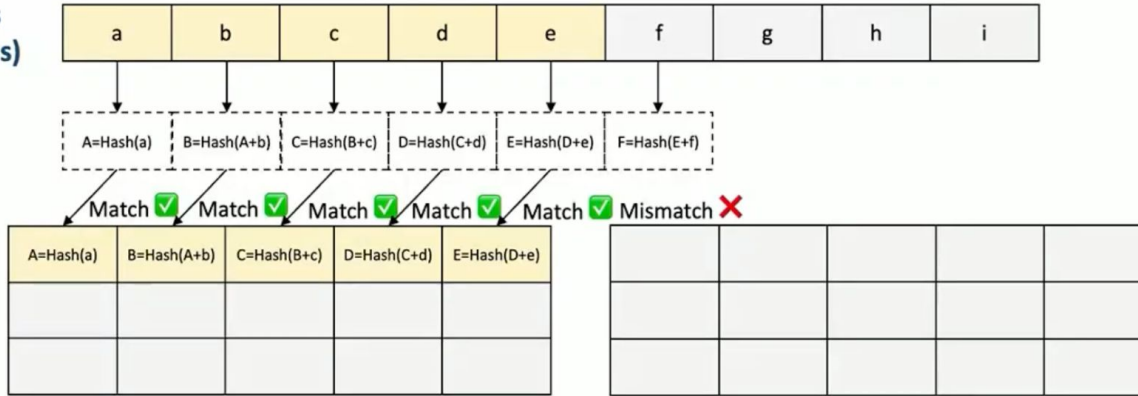
Prefill
Instance

Decoding
Instance

Mooncake Store: Cache of KVCache



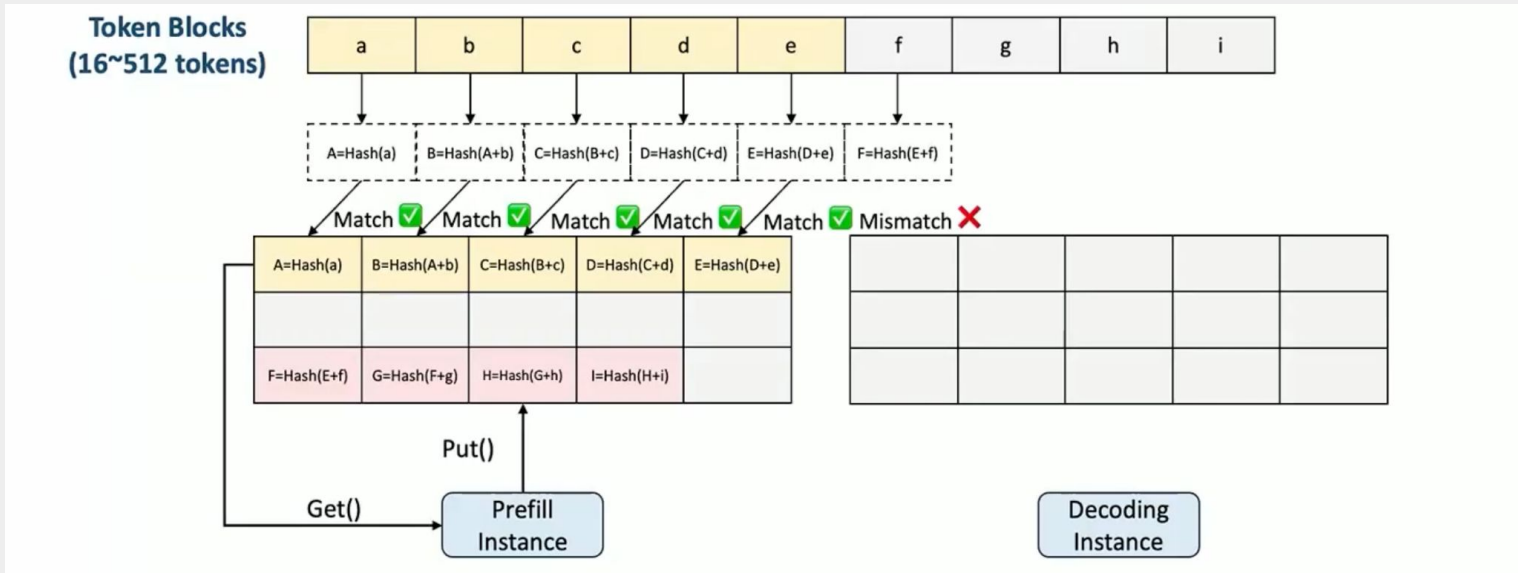
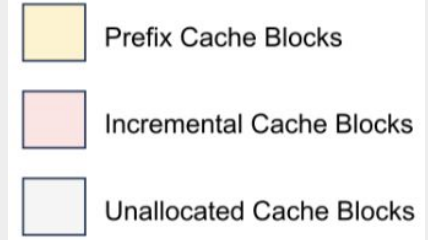
Token Blocks
(16~512 tokens)



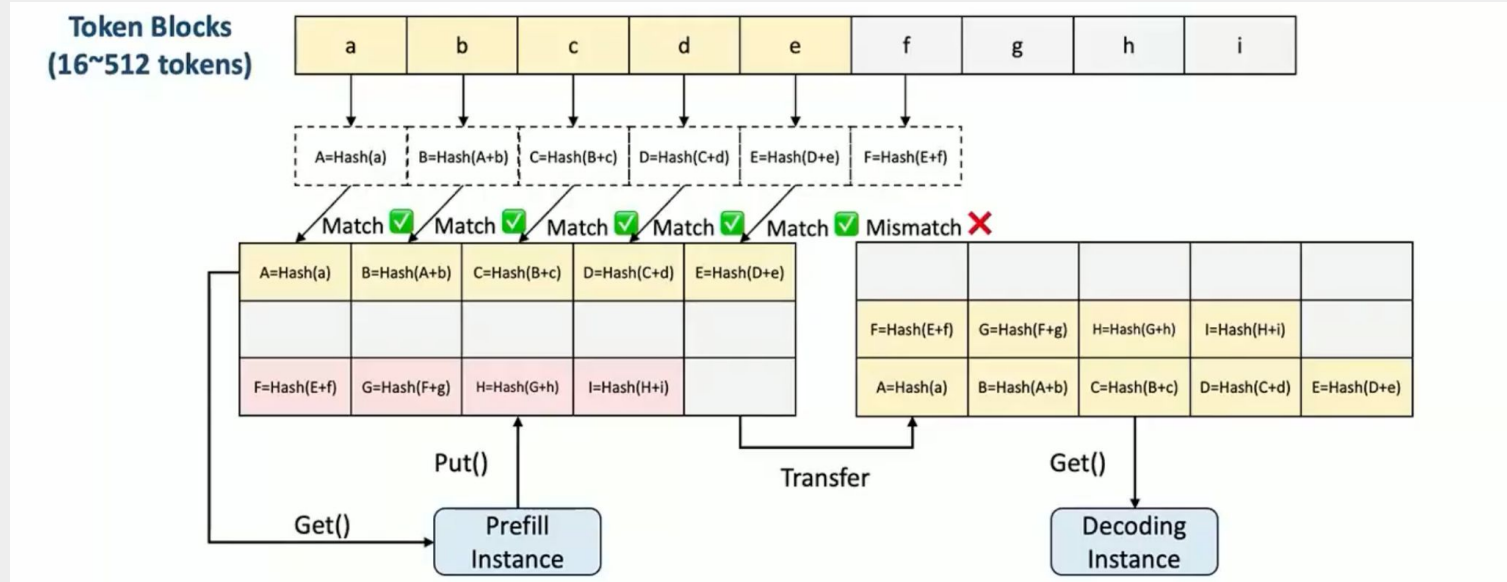
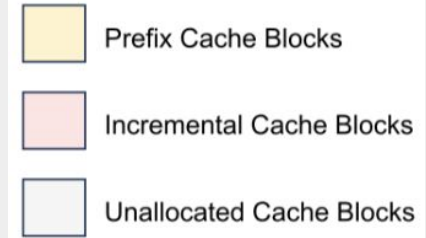
Prefill
Instance

Decoding
Instance

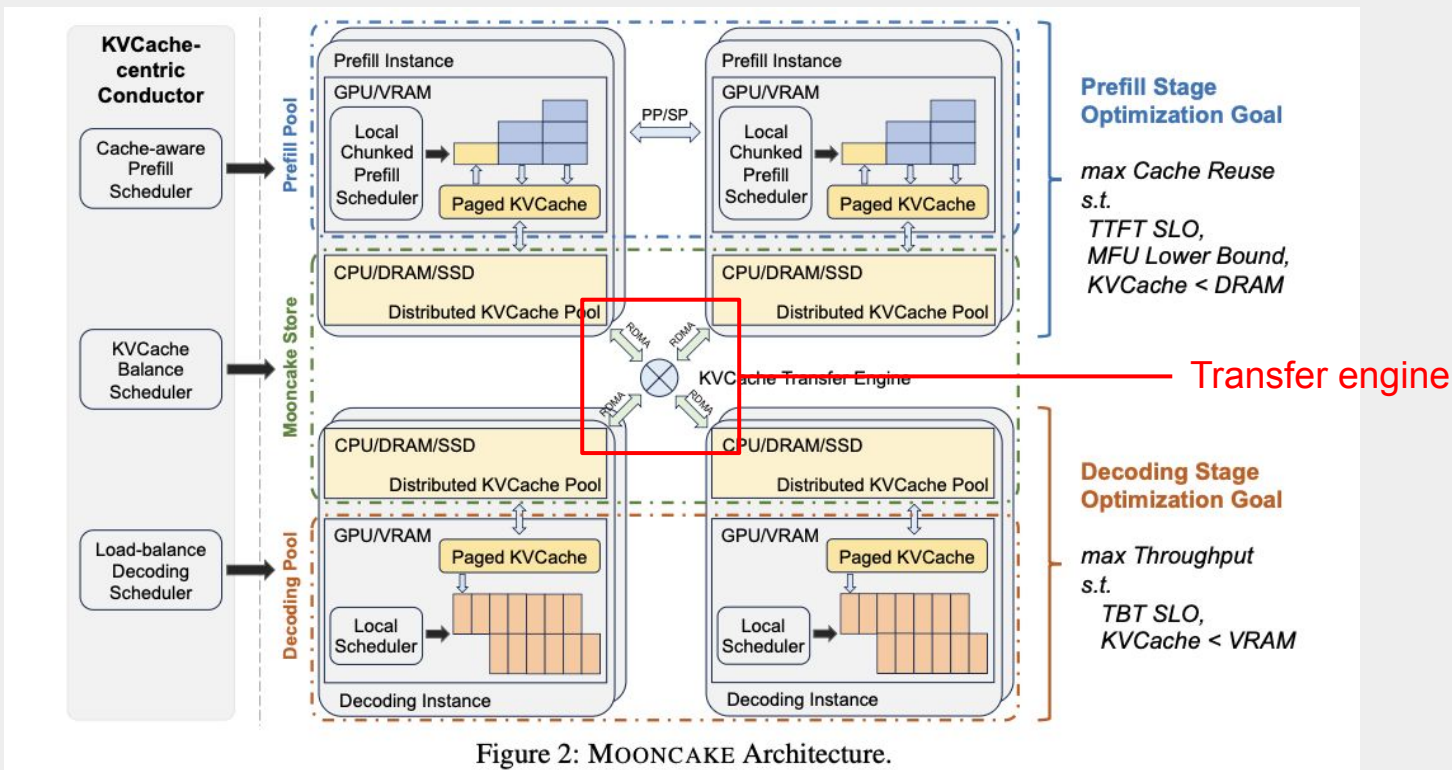
Mooncake Store: Cache of KVCache



Mooncake Store: Cache of KVCache



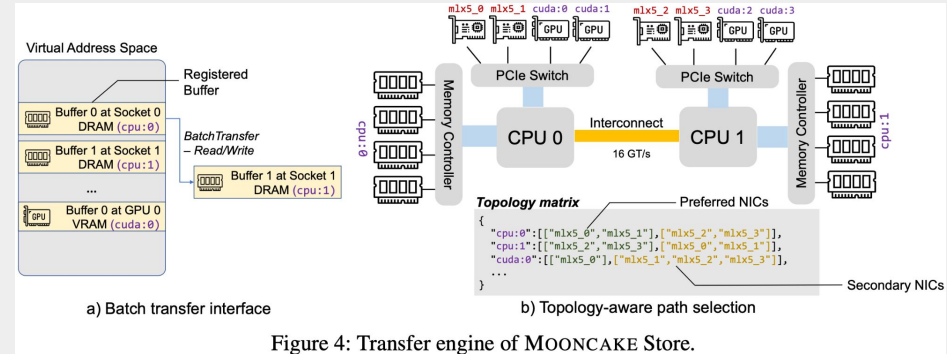
Mooncake: Architecture



Mooncake Store: Transfer Engine

1. Distribute transfer tasks across multiple RDMA NIC devices
2. Abstract complexities of RDMA connection management from APIs
3. Handle network failures

- API Interface/Transfer Engine
 - Topology-aware path selection
 - Failure handling
- Remote Direct Memory Access (RDMA)



Mooncake: Architecture

Conductor

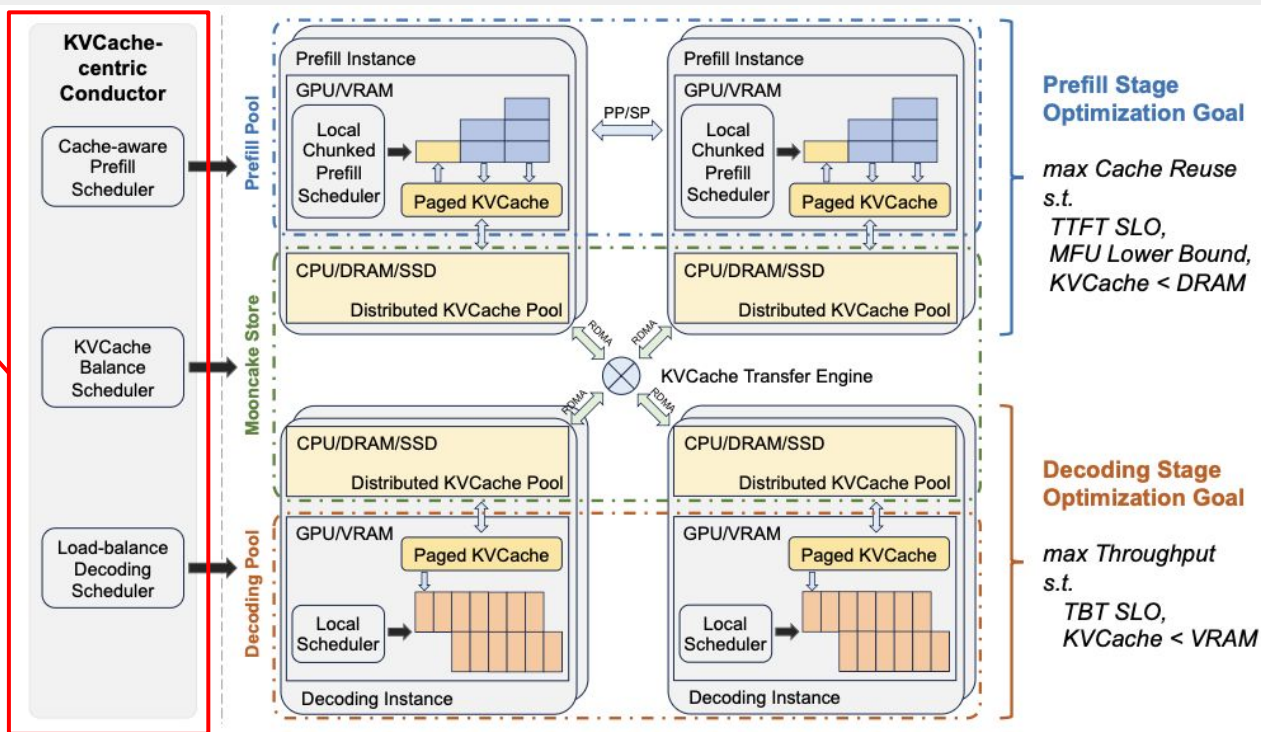


Figure 2: MOONCAKE Architecture.

Scheduling Algorithm

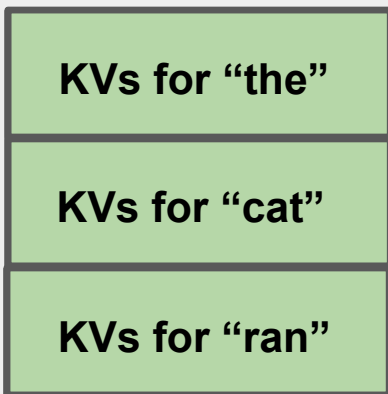
- Most LLM serving systems route prefill requests purely based on load (the number of ongoing requests per instance)
- Mooncake introduces a **global cache-aware** scheduling algorithm that takes into account a few additional factors
 - **Goal:** Select a prefill and decode instance to route this request to.
 - **How?** Estimate TTFT for all prefill instances and assign to the instance with the lowest TTFT.
 - **How is TTFT estimated on each instance?** By taking into account the:
 - Size of the prefix match and the incremental prefill time required
 - Queuing time
 - Transfer time (if transferring KVCache from another instance)

Scheduling Algorithm, Part 1

Step 1: For every new request, find the length of the prefix match on each prefill instance.

Decode Instance 1

previously handled prefill request “the cat ran”



New Request: the cat slept

Prefix Match Length = 2

repeat this for all other nodes and
keep track of longest match

Scheduling Algorithm, Part 2

Step 2: Estimate the prefill execution time of the request on each instance and selects the instance with the lowest TTFT.

```
for instance  $\in P$  do
  if  $\frac{best\_len}{instance.prefix\_len} > kvcache\_balancing\_threshold$  then
    prefix_len  $\leftarrow best\_len$ 
    transfer_len  $\leftarrow best\_len - instance.prefix\_len$ 
     $T_{transfer} \leftarrow \text{EstimateKVCacheTransferTime}(transfer\_len)$ 
  else
    prefix_len  $\leftarrow instance.prefix\_len$ 
     $T_{transfer} \leftarrow 0$ 
   $T_{queue} \leftarrow \text{EstimatePrefillQueueTime}(instance)$ 
   $T_{prefill} \leftarrow \text{EstimatePrefillExecutionTime}(\text{len}(R.prompt\_tokens), prefix\_len)$ 
  if  $TTFT > T_{transfer} + T_{queue} + T_{prefill}$  then
     $TTFT \leftarrow T_{transfer} + T_{queue} + T_{prefill}$ 
     $p \leftarrow instance$ 
```

We compute the ratio of the length of the longest prefix to the prefix length on this instance.

- If this ratio exceeds a predefined threshold, transfer the KVCache from the “best” node and add this transfer time to the TTFT.
- Otherwise, prefill all tokens after the local prefix on this instance (transfer time = 0).

At the end of this loop, **p** will be the prefill instance with the lowest TTFT.

Scheduling Algorithm, Part 3

Step 3: Pick a decode instance and verify that the SLOs are met. Finally, (if necessary) initiate the KVCache transfer between the “best” node and the selected node.

Scheduling Algorithm Estimations

How are queuing, execution, and transfer time estimated?

- **Execution Time:** polynomial regression model that takes into account the prefix length and overall request length
- **Queuing Time:** aggregate the prefill times of all queued request
- **Transfer Time:** based on network bandwidth and size of transferred data (doesn't take into account network congestion)

Scheduling Algorithm Evaluation

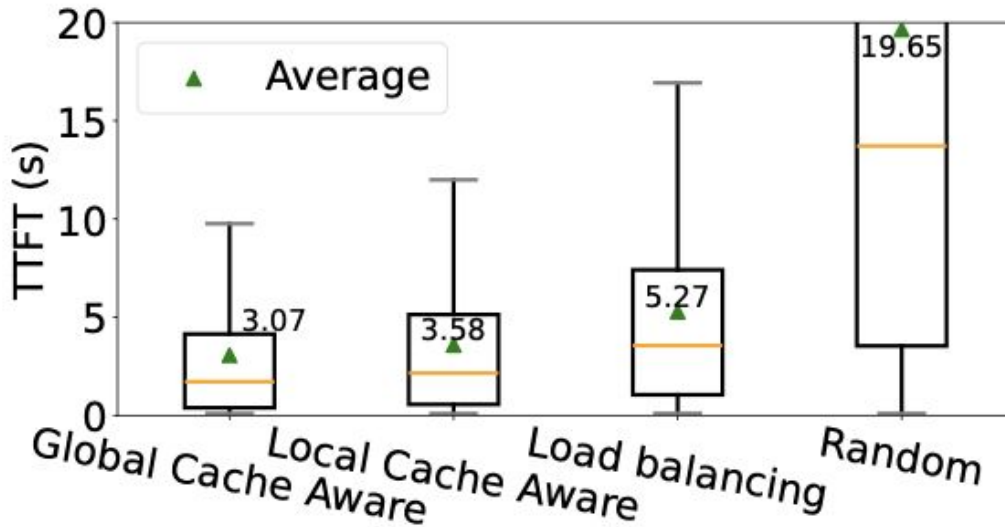


Figure 5: The prefill scheduling experiment.

Cache Load Balancing

- Some KVCache blocks may be “hot”/in high-demand (like the blocks corresponding the system prompt for a model)
- These blocks need to be replicated across instances to prevent congestion
- This replication happens **naturally** as a result of the scheduling algorithm, since requests aren’t always routed to the instance with the longest prefix match
 - When a request lands on a different instance, the shared prefix’s KVCache is either (1) transferred from the best node or (2) recomputed locally
 - In either case, the KVCache blocks are now replicated across instances

Evaluation

Evaluated Mooncake on LLAMA3-70B model to answer the following two questions:

They compare Mooncake against:

- **vLLM**: state-of-the-art serving system with continuous batching and PagedAttention, does not separate prefill and decode clusters
- **vLLM with chunked prefill**
- **vLLM with prefix caching** (but local to each instance)

Across three different workloads:

- Conversation Workload
- Tool & Agent Workload
- Synthetic Workload

Evaluation

Conversation Workload

- Based on a one-hour sample of multi-turn conversations from an inference cluster
- Represents one of the most prevalent/common applications of LLMs

	Conversation	Tool&Agent	Synthetic
Avg Input Len	12035	8596	15325
Avg Output Len	343	182	149
Cache Ratio	40%	59%	66%
Arrival Pattern	Timestamp	Timestamp	Poisson
Num Requests	12031	23608	3993

Table 2: Workload Statistics.

Cache Ratio - the % of prompt tokens that can be served from the KVCache (instead of being recomputed)



Evaluation

Tool and Agent Workload

- LLMs are increasingly being used as tools/agents to perform certain tasks
- These agents typically have length system prompts (high cache ratio!)

	Conversation	Tool&Agent	Synthetic
Avg Input Len	12035	8596	15325
Avg Output Len	343	182	149
Cache Ratio	40%	59%	66%
Arrival Pattern	Timestamp	Timestamp	Poisson
Num Requests	12031	23608	3993

Table 2: Workload Statistics.

Cache Ratio - the % of prompt tokens that can be served from the KVCache (instead of being recomputed)



Evaluation

Synthetic Dataset

- Heterogeneous workload featuring short conversations, agent/tool calls, long text summarization, and Q/A from publicly available datasets
- High cache ratio, but cache hits are quite dispersed
 - So a large cache capacity is needed to take advantage

	Conversation	Tool&Agent	Synthetic
Avg Input Len	12035	8596	15325
Avg Output Len	343	182	149
Cache Ratio	40%	59%	66%
Arrival Pattern	Timestamp	Timestamp	Poisson
Num Requests	12031	23608	3993

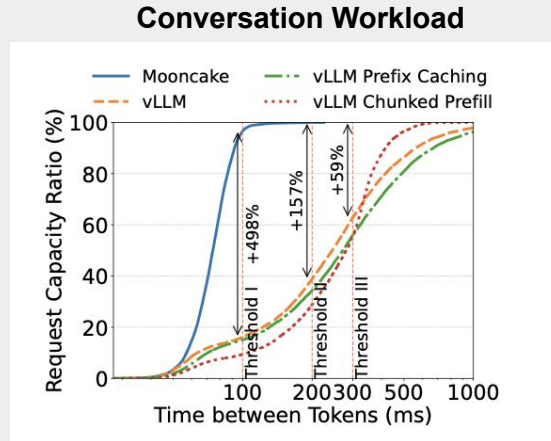
Table 2: Workload Statistics.

Cache Ratio - the % of prompt tokens that can be served from the KVCache (instead of being recomputed)



Effective Request Capacity Mooncake

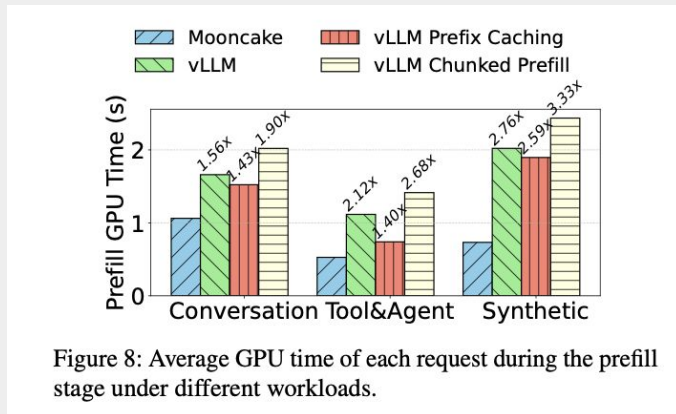
- Effective Request Capacity: the percentage of incoming requests that the system can handle while still meeting its SLOs



- Separating prefill and decode into distinct clusters prevents long context prefills from interfering with the decode phase (i.e. TBT)

Average Prefill Time

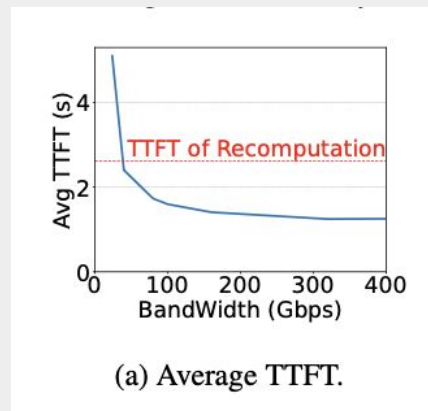
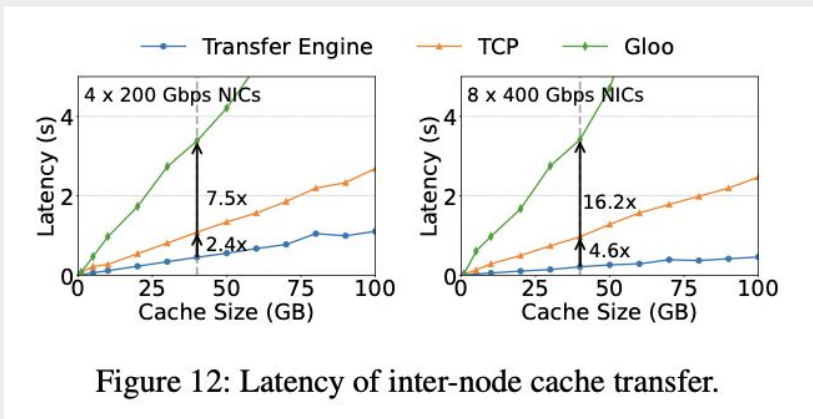
- Mooncake's prefill time is lower than the other three systems on all three workloads



- Its results relative to vLLM Prefix Caching show the benefit of a **global** KV cache, where each node can access the KVs computed by any other node

Transfer Engine Performance

- Mooncake's transfer engine is what makes global KVCache sharing feasible
 - Found to have significantly lower latency than the alternatives
- Having sufficiently large bandwidth (> 100 Gbps) is crucial for system performance



Weaknesses

- Limited discussion of fault tolerance (aside from transfer engine)
 - What happens if the conductor node goes down? Is it replicated somehow?
- No evaluation on workload with low prefix cache ratio
 - Examples: single-turn Q/A, image/multimodal processing
- Hardware requirements make it impractical for small organizations/deployments

Future Work

- Authors are currently working on integrating Mooncake with vLLM

Conclusion

Conclusion

- NanoFlow
 - NanoFlow is a serving framework that boosts LLM throughput by overlapping compute/memory/network bound operations within each GPU using fine-grained nano-batching and automated pipeline scheduling.
- Mooncake
 - Introduces a disaggregated LLM serving system that (1) separates prefill and decode phases into dedicated clusters and (2) leverages a global KVCache to maximize throughput while meeting strict TTFT and TBT SLOs.

Appendix

Appendix A: Memory, Compute, Network Latency

$$T_{mem} = \frac{MemSize}{MemBW}$$

$$T_{Compute} \approx \frac{2B_{Dense} \cdot P_{Model}}{Compute}$$

$$T_{net} \approx 4 \cdot \frac{(N_{GPU} - 1)B_{Dense}D_{model}S_{type}L}{NetBW}$$

Appendix B: Request Scheduling

- Batch formation
 - NanoFlow prioritizes unfinished decode requests and chunks prefill requests
 - As prefill requests finish and become decode requests, new prefill requests join the batch to keep token count stable, naturally balancing both phases over time
 - NanoFlow predicts peak future memory usage and prefills new requests only if predicted memory usage stays within GPU limits
- Asynchronous scheduling
 - In most serving systems, the CPU scheduler only removes finished requests and refills the batch after the GPU finishes an iteration, leaving the GPU briefly under-utilized
 - NanoFlow overlaps CPU scheduling with GPU work by forming the next batch before the current one ends; it only detects EOS tokens one iteration later but keeps the GPU continuously busy
 - Since the average decode length is over 100 tokens, the cost of processing one extra token is negligible (<1%) compared to the benefit of hiding batch formation overhead

Appendix C: KV Cache Management

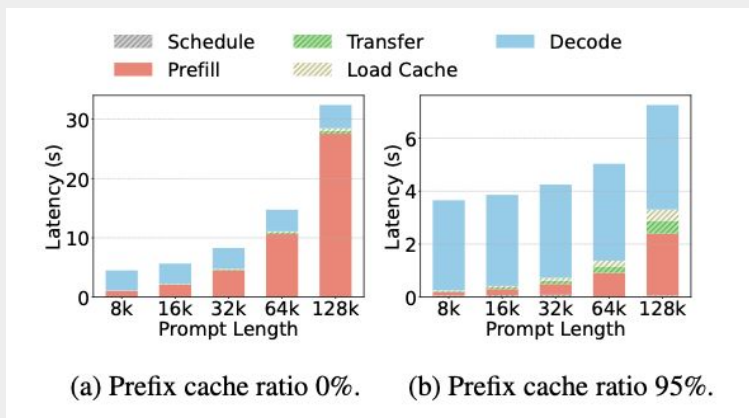
- Simultaneous offloading
 - NanoFlow offloads each token's KV-cache immediately after KQV generation in every layer, overlapping data transfer with ongoing computation instead of waiting for the entire request to finish.
- Host KV-cache management
 - NanoFlow manages its CPU and SSD cache using an LRU policy, evicting KV-caches to SSD when CPU memory is full and reloading them from either CPU or SSD when a conversation resumes
- KV-cache loading and scattering
 - NanoFlow copies KV-cache data into a contiguous GPU buffer before scattering it to fragmented pages, boosting host-to-device transfer bandwidth by 7–10×

Appendix D: NanoFlow Experimental Setup

- LLaMA-2-70B Model
 - FP16 weights and activations
- 8 × A100 80GB SXM GPUs
 - Interconnected via NVLink
- Baselines
 - vLLM: SOTA serving system delivering high throughput with pagedAttention and chunked prefill
 - DeepSpeed-FastGen: Serving framework developed by Microsoft by composing prefill with decode requests
 - TensorRT-LLM: high-performance LLM inference engine built upon NVIDIA's TensorRT SDK
- Datasets
 - Splitwise: Conversation trace collected from a real production environment at Microsoft
 - LMSYS-Chat-1M: Large-scale dataset with 1 million real-world conversations from 25 different LLMs
 - ShareGPT is a dataset with conversations collected from the ShareGPT API

Appendix E: Mooncake E2E Latency

- Prefix caching significantly reduces proportion of time spent doing prefill
- Mooncake adds **Schedule** and **Transfer** overhead during prefill, but this overhead is smaller than the reduction achieved via prefix caching
 - The steps are done asynchronously while inference is happening



Appendix F: Prefix Caching

Notation	Description	Value
l	Num layers	80
d	Model dimension	8192
a, b	Constant coefficients in Equation 1	4, 22
gqa	Num q heads / Num kv heads	8
s	Tensor element size	2 B (BFloat16)
G	GPU computation throughput	8×312 TFLOPS
B_{h2d}	Host-to-device bandwidth	128 GB/s
B_{nic}	NIC bandwidth	800 Gbps
n, p	Prompt and matched prefix length, respectively	

$$q[p : n], k[p : n], v[p : n] = \text{MLP}(\text{hidden}[p : n])$$

→ Compute KQV for uncached tokens

$$k[1 : n], v[1 : n] \leftarrow \text{KVCache} + (k[p : n], v[p : n])$$

→ Compute KV

$$o[p : n] = \text{Attention}(q[p : n], k[1 : n], v[1 : n])$$

→ Compute attention for new tokens

$$\text{KVCache} \leftarrow (k[1 : n], v[1 : n])$$

→ Update KVCache

Appendix G: More Storage for Less Computation

Notation	Description	Value
l	Num layers	80
d	Model dimension	8192
a, b	Constant coefficients in Equation 1	4, 22
gqa	Num q heads / Num kv heads	8
s	Tensor element size	2 B (BFloat16)
G	GPU computation throughput	8×312 TFLOPS
B_{h2d}	Host-to-device bandwidth	128 GB/s
B_{nic}	NIC bandwidth	800 Gbps
n, p	Prompt and matched prefix length, respectively	

$$flops(n) = l \times (an^2d + bnd^2)$$

- Reusing KVCache reduces computation cost by $flops(n)$

$$\frac{B}{G} > \frac{2ds}{gqa \times (apd + bd^2)}$$

- Only reuse KVCache under this scenario

Appendix H: Mooncake Scheduling Algorithm

Algorithm 1 KVCache-centric Scheduling Algorithm

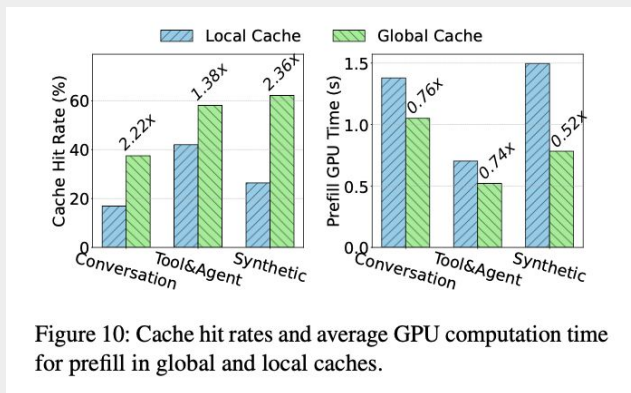
Input: prefill instance pool P , decoding instance pool D , request R , cache block size B .

Output: the prefill and decoding instances (p, d) to process R .

```
1:  $block\_keys \leftarrow \text{PrefixHash}(R.prompt\_tokens, B)$ 
2:  $TTFT, p \leftarrow \text{inf}, \emptyset$ 
3:  $best\_len, best\_instance \leftarrow \text{FindBestPrefixMatch}(P, block\_keys)$ 
4: for  $instance \in P$  do
5:   if  $\frac{best\_len}{instance.prefix\_len} > \text{kvcache\_balancing\_threshold}$  then
6:      $prefix\_len \leftarrow best\_len$ 
7:      $transfer\_len \leftarrow best\_len - instance.prefix\_len$ 
8:      $T_{transfer} \leftarrow \text{EstimateKVCacheTransferTime}(transfer\_len)$ 
9:   else
10:     $prefix\_len \leftarrow instance.prefix\_len$ 
11:     $T_{transfer} \leftarrow 0$ 
12:   $T_{queue} \leftarrow \text{EstimatePrefillQueueTime}(instance)$ 
13:   $T_{prefill} \leftarrow \text{EstimatePrefillExecutionTime}(\text{len}(R.prompt\_tokens), prefix\_len)$ 
14:  if  $TTFT > T_{transfer} + T_{queue} + T_{prefill}$  then
15:     $TTFT \leftarrow T_{transfer} + T_{queue} + T_{prefill}$ 
16:     $p \leftarrow instance$ 
17:  $d, TBT \leftarrow \text{SelectDecodingInstance}(D)$ 
18: if  $TTFT > TTFT\_SLO$  or  $TBT > TBT\_SLO$  then
19:   reject  $R$ ; return
20: if  $\frac{best\_len}{p.prefix\_len} > \text{kvcache\_balancing\_threshold}$  then
21:    $\text{TransferKVCache}(best\_instance, p)$ 
22: return  $(p, d)$ 
```

Appendix I: Global Cache Versus Local Cache

- Compared the global cache setup to a local cache set up where:
 - Each node has 3M token capacity, but can only access its local KVs
 - Requests are always routed to the instance with the longest prefix



Global cache leads to:

- Up to **136%** increase in cache hit rate, maximum reduction in prefill time of **46%**

Archive